

Reuse, but Verify: Certified Maintenance of Table Cells Computed by Large Language Models under Prompt Edits

ARJUN LOHAN¹

¹University of Southern California, Los Angeles, CA 90089 USA (e-mail: lohan@usc.edu)

Corresponding author: Arjun Lohan (e-mail: lohan@usc.edu).

This work received no external funding.

ABSTRACT Modern tables let users define columns in natural language: a language model computes each cell from a versioned prompt over its row. When the prompt is edited, systems either recompute every cell or reuse stale ones, with unbounded error. We present semantic incremental view maintenance (sIVM), which certifies the cached cells that may be reused after a prompt edit. It groups cells by cached value into strata and bounds each certified stratum's false-reuse rate by a user-chosen error budget with stated confidence. It samples each stratum on a doubling schedule and bounds its flip rate with an exact finite-population bound; a strict mode bounds the error of the reused cells alone. An impossibility floor follows: with the prompt unedited, no stratum certifies below its own rate of self-disagreement. On two public corpora (28,000 labeled cells), a formatting-only edit certifies 88.4% call savings (88.1% mean over 1,000 replications) at an error budget of 0.2 with 3.20% realized false-reuse; at deployment scale, 90 fresh calls certify reuse of 81,424 of 89,184 cells. Across the calibration grid, a certificate exceeds its budget with probability at most 2.6% at the least favorable population, against a nominal 5%. A second certification 55 days later measured drift of the served model and re-certified the same budgets. At least one edit certifies on 6 of 6 further model families; which edits are benign varies by family. The system is released under a source-available noncommercial license, with labels and a benchmark of versioned prompt-edit pairs.

INDEX TERMS Caching, confidence bounds, incremental view maintenance, large language models, materialized views, prompt engineering, query processing.

I. INTRODUCTION

Interactive artificial intelligence (AI) tables turn a natural-language prompt into a *materialized view*: a column $V = (\pi, \mathcal{M}, \theta)$ with prompt π , model $\mathcal{M}$, and pinned (held fixed) decode parameters θ . The view is materialized as cells c_i over rows $r_1, \dots, r_n$, each cell one draw from the model's output distribution $f_V(r_i)$. The category is now a product surface rather than a research proposal. Google Sheets evaluates an `AI()` formula per row and offers AI columns in tables [1], Airtable computes AI fields per record [2], Notion autofills database properties with a model [3], and Snowflake Cortex applies `AI_COMPLETE` and `AI_CLASSIFY` to table columns in SQL [4]. Microsoft's Excel `COPILOT` function shipped as a preview and was withdrawn in September 2026, a measure of how quickly this surface is moving [5]. These views are edited constantly: users iterate on prompts the way they iterate on spreadsheet formulas. Every edit invalidates the cache by construction, and today's systems invalidate it

wholesale: an edit to an 89,184-row column triggers 89,184 fresh model calls, or the user silently keeps stale values with no bound on how wrong they are.

a: Running example.

One column from our evaluation recurs throughout. Over 89,184 Stack Overflow (SO) developer profiles, a Boolean AI column asks “*Is this person a data-platform specialist?*”, judged from each profile's role, databases, platforms, and tools. Its owner edits the prompt to insert two paragraph breaks and lowercase one word, changing no content word (the *formatting-only* edit of Section V; Section VI lists every edit we study). Recomputing costs 89,184 model calls. Keeping every cached value silently carries the edit's flip rate into the table: the share of cells a recompute would change, 6.3% on this pair. sIVM instead groups the cached cells by their cached value (the two *strata*, cached-TRUE and cached-FALSE), recomputes a small seeded sample from each, and

reuses a stratum only when a statistical bound on its flip rate clears the user's error budget. On this column at deployment scale, 90 fresh calls certify reuse of 81,424 of 89,184 cells. They do so on the stored draws of the August run, replayed under the exact bound, and again in a live run 55 days later, against a model that had drifted in the meantime.

Recent semantic data systems bound the error of *one-shot* query execution against a reference model whose answers are taken as correct (cascades, semantic operators, approximate processing with guarantees). Verified semantic caches bound reuse under a *fixed* definition (Section VII names both lines). Neither answers the maintenance question an interactive table poses: *after this specific edit $\pi \rightarrow \pi'$, which cached cells are still valid, and with what guarantee?*

A. WHY INTUITION FAILS HERE

Three measurements reframe the problem (Section V). First, large language model (LLM) cells are stochastic even at temperature zero [6], [7]: the same prompt on the same row disagrees with itself 5.0% of the time on our Boolean workload and 9.5% at default temperature (Section V gives the measurement behind each figure). “Did the edit change this cell?” is therefore only meaningful relative to a noise floor, the rate at which the model disagrees with itself under the unedited prompt. Second, edit direction is not monotone: under a strictly scope-widening edit, 45.7% of cached-TRUE cells flipped ([39.0, 52.7] over 199 such cells), refuting the folk lemma that widening preserves positives. Third, an edit's surface class is not a safe predictor of its semantic impact. On the same column, a synonym paraphrase written to preserve meaning flipped 8.9% of cells where a formatting-only edit flipped 6.3%. On a candidate-profile column from a second corpus, a formatting-only edit flipped 12.3% against 12.4% for a deliberate criteria change, rates that are indistinguishable. Neither pair orders the way the edits' labels suggest. The study's five edits are author-written, one per class per column, so they show that class can fail to predict flip rate, not how often it does. The certifier below never consults the class, and Section V shows the same edit's flip rate varying widely across model families. Both comparisons hold the column fixed deliberately: on the two column types we study, the column's own stability moves the flip rate more than the edit does.

Sound reuse therefore has to be *measured*: sample a few fresh cells, statistically bound the flip rate of the rest, and reuse only what clears the user's error budget.

B. CONTRIBUTIONS

- 1) **Problem formalization** (Section II): certified cross-version reuse for stochastic prompt-defined views. Two estimands (the population quantities a guarantee is about) are well-defined under decode pinning. One is the realized false-reuse rate over the cells a certified stratum presents, which we certify by default. The other is the same rate over the reused cells alone, which the strict mode certifies at a correspondingly lower thresh-

old. An impossibility floor follows: with the prompt unedited, a budget α below a stratum's self-flip rate is not certifiable.

- 2) **The sIVM procedure** (*semantic incremental view maintenance*; Section III): a frozen stratification, an adaptive sample, and a futility rule. The stratification is by cached value, optionally refined by how strongly the edit's changed terms match each row. Sampling follows a doubling schedule of *looks* (points at which the sample is inspected). The futility rule gives up on a stratum whose sample already exceeds the budget. Certification is per-stratum with an exact finite-population confidence bound, and the procedure offers two guarantee targets (presented cells; strict reuse set). Deciding at several looks is *peeking*; the validity theorem survives it by splitting the failure probability δ across the looks. Any frozen stratifier affects power only, never validity, where power is the chance of certifying a stratum that does meet the budget. The bound is a pluggable component: the exact bound is proven for the without-replacement sampling the procedure performs, and four alternatives (two empirical-Bernstein bounds, the binomial Clopper–Pearson bound, and a betting confidence sequence) are measured inside the same procedure.
- 3) **A production implementation** (Section IV) inside a working AI-table system, not a simulation harness, released under a source-available noncommercial license. The system has a per-cell runner, a content-hash cache, a cost ledger, and certificates (the persisted records of what cleared the budget), which the user interface (UI) surfaces.
- 4) **An empirical anatomy** (Section V) of one primary model on two real corpora with 28,000 labeled cells. It measures the noise floor, the savings frontier over the error budget, and minority-stratum volatility, along with how much of it majority voting at write time removes. It includes a stratifier ablation (refinement leaves validity intact but costs power on this benchmark), a calibration study of how often a certificate exceeds its budget, against the nominal δ , a check of the cross-row independence assumption, and a model-family study on 6 further families. It also reports a re-certification 55 days after the deployment run. The re-certification measured drift of the served model as a higher flip rate against the same cache and showed the certifier pricing it. The budgets $\alpha=0.2$ and $\alpha=0.1$ certify again, the second at a look deeper than on the August draws, and $\alpha=0.05$, on which the August draws are silent, is refused. A free-text pilot (Section VIII) calibrates a judge's error on hand labels and folds it into the budget.
- 5) **A reusable benchmark** (Section VI): versioned prompt-edit pairs with full ground-truth flip labels over public corpora redistributable under the Open Database License (ODbL) and the MIT License, enabling like-for-like evaluation of future maintenance systems.

The remainder of this article is organized as follows. Section II formalizes certified reuse and its two guarantee targets. Section III presents the sIVM procedure and its validity theorem. Section IV describes the production system it runs in. Section V reports the evaluation. Section VI describes the released benchmark of versioned prompt-edit pairs. Section VII positions the work against prior art. Section VIII states limitations, and Section IX concludes. Appendix A lists the mechanical guards on the manuscript's numbers and Appendix B the bound ablation configuration by configuration.

II. PROBLEM

a: Terminology.

Four words recur. A *stratum* is a group of cached cells fixed before any sampling (by default, the cells that share a cached value). A *look* is a point in the sampling schedule at which the sample drawn so far is inspected and a decision may be taken. A stratum's *floor* is its self-flip rate, the rate at which the model disagrees with its own earlier answer when nothing has changed. Under the unedited prompt no budget below the floor is certifiable, and the impossibility-floor paragraph below says how far that carries to edits. A *certificate* is the persisted record that a stratum cleared the budget, with the evidence it cleared it on. Three verbs recur with them. A cell *flips* when a fresh draw under the edited prompt disagrees with its cached value. A stratum that is not certified is *refused*, and its unsampled cells are recomputed. Sampling stops for *futility* when the sample's own flip rate already exceeds the budget. A few statistical terms recur as well. An *oracle draw* (or oracle call) is one fresh model call under the edited prompt for one row. An *estimand* is the population quantity a guarantee is about. *Power* is the chance that a stratum which does meet the budget is in fact certified. *Pinned* means held fixed for the duration of a certification (the model, the decode parameters, the stratification). The *Bonferroni* split divides a failure probability among several tests so that their failure probabilities cannot add up past it. Table 1 collects the symbols.

b: Views and cells.

An AI column is a view $V = (\pi, \mathcal{M}, \theta)$: a prompt template π with a typed output contract, a model $\mathcal{M}$, and pinned decode parameters θ . For row r_i , the bound prompt $\pi(r_i)$ interpolates only the fields $\beta(\pi)$ that π references; the stored cell c_i is *one draw* from the output distribution $f_V(r_i)$. Even at temperature zero this distribution is not degenerate: providers are nondeterministic across calls, as prior measurements of hosted models document [6], [7], in part because the floating-point reduction order inside serving kernels changes with the co-batched load [8], [9]. Every quantity below is therefore defined over draw randomness.

c: The maintenance event.

The user edits the definition, $V \rightarrow V' = (\pi', \mathcal{M}, \theta)$. For each cached cell define the flip indicator $F_i = \mathbf{1}[c_i \neq X'_i]$, where $X'_i \sim f_{V'}(r_i)$ is a fresh draw under the new definition

TABLE 1. Notation.

Symbol	Meaning
$V = (\pi, \mathcal{M}, \theta)$	a view: prompt template, model, pinned decode parameters; $V' = (\pi', \mathcal{M}, \theta)$ after the edit
r_i, c_i	row i and its cached cell, one draw from $f_V(r_i)$
X'_i, F_i, p_i	a fresh draw under V' ; the flip indicator $\mathbf{1}[c_i \neq X'_i]$; its mean
$\equiv$	the column's equivalence relation (exact match here)
j, n_j, s_j, p_j	a stratum, its size, its self-flip rate (floor), its flip rate (the mean of p_i over $i \in j$)
$\Delta, \beta(\pi)$	the prompt delta (content terms added or removed); the row fields π interpolates
$\mathcal{S}, m_j$	the sampled (freshly recomputed) cells; the number of them in stratum j
$\mathcal{R}$	the reuse set: cached cells kept under a certificate
L_j, t, n_t	stratum j 's look schedule, a look in it, and the cells drawn by that look; n_0 is the first look and ℓ the most looks allowed
N, n, k	in the bound: the stratum's size ($N = n_j$), the cells drawn at a look, and the flips among them
$u_{j,t}$	the upper confidence bound on stratum j 's flip rate at look t
α, δ	error budget on the false-reuse rate; upper bound on the probability that any certificate is wrong
K, δ_j, δ'	number of strata; the per-stratum failure probability δ/K ; the per-look level $\delta_j/ L_j $
M_j, k_j	stratum j 's realized flip count; the flips observed in its sample
$\text{FR}_j^{\text{pres}}$	realized false-reuse of stratum j over the cells it presents
$\text{FR}_j^{\text{reuse}}$	the same over its reused cells alone

and $\equiv$ is the column's equivalence relation (exact match for the Boolean and select columns studied here). The indicator F_i is Bernoulli with unknown mean $p_i = \Pr[c_i \neq X'_i]$. That mean deliberately folds together the edit's semantic effect and decode noise on both sides: it answers the operational question of whether recomputing cell i now would return a value different from the one shown.

d: Estimand and guarantee.

A maintenance policy partitions the cached cells into a sampled set $\mathcal{S}$ (freshly recomputed), a reuse set $\mathcal{R}$ (stale values kept), and a recompute set. Two error rates matter, and they differ. The *presented-cells* rate of a certified stratum j is

$$\text{FR}_j^{\text{pres}} = \frac{1}{n_j} \sum_{i \in j \setminus \mathcal{S}} F_i :$$

of all n_j cells the user now sees from that stratum, the fraction that would flip if recomputed (sampled cells are fresh and contribute zero). That zero is a convention of the single-draw estimand, not a property of recomputed cells: a recomputed cell is itself one draw, and a further draw would disagree with it at about the stratum's self-flip rate. A flip is any

disagreement with one fresh draw, decode noise included, as the previous paragraph defines it. The estimand charges reuse for exactly what the user would see change on recomputation, which is the operational question. The impossibility floor below is the price of that choice: an excess-over-floor target, $p_j - s_j$, would certify the identity edit for free. We deliberately do not adopt that target, because a user who recomputes sees one draw and lives with it. The *reuse-set* rate

$$FR_j^{\text{reuse}} = \frac{1}{n_j - m_j} \sum_{i \in j \setminus S} F_i$$

normalizes by the $n_j - m_j$ reused cells alone and is never smaller. A certificate bounds the stratum's whole flip count, and hence FR_j^{pres} , but not FR_j^{reuse} : the remainder is adversely selected, because certification happens exactly when the sample looked clean. sIVM therefore certifies the presented-cells estimand by default, and offers a strict reuse-set mode that certifies at the lower (*deflated*) threshold $\alpha(n_j - m_j)/n_j$, which bounds FR_j^{reuse} by conservation of flips. On the *headline pair*, the SO formatting-only edit of the running example, at $\alpha=0.2$ the two rates read 3.20% and 3.28%, the latter with a 95% confidence interval (CI) of [2.5, 4.2] (Table 2). Theorem 1 states both guarantees for the realized rates of this run, not for their expectations. Table 2 reports both realized rates for the main results. All guarantees are per-stratum and therefore hold simultaneously for every union of certified strata, not only the aggregate (baseline B2 in Section V, aggregate-only certification, shows why the aggregate alone is operationally misleading).

sIVM makes no decision about an individual cell. Whether one cached cell would change under the edited prompt can be learned only by recomputing it, which spends exactly the call that reuse would save. The cheapest per-cell signal, prompt similarity, predicts flips at chance across a definition change (baseline B1 in Section V). The signals that do carry information, the cached value and how strongly the edit's changed terms match the row (the interaction score of Section III), separate groups of cells, not single cells, and enter as strata. The unit of decision is therefore the stratum: an unsampled cached cell is reused if and only if the upper confidence bound on its stratum's flip rate clears the budget at a look the schedule reaches. That is the correctness criterion for reuse, and it is a statement about recomputation, not about ground truth. A certificate says that recomputing the stratum now would change at most an α share of its cells, with probability at least $1 - \delta$, not that the cells are right.

e: Reading the budget.

A certificate says: with probability at least $1 - \delta$ over this run's sampling and oracle draws, at most an α fraction of the certified stratum's cells would flip if recomputed (default mode), or at most an α fraction of its reused cells alone (strict mode). Here α is that share, and δ bounds the probability that any certificate of the run is wrong at all. Both are the user's to set, and the right values are workload-specific: a recruiter screen that a person reviews row by row tolerates a looser α than a column that feeds a downstream aggregate, and the

strict mode exists for the latter. The realized error under a certificate is typically far below α (3.20% against $\alpha=0.2$ on the headline pair, Table 2), because the bound must cover the worst population compatible with the sample. The frontier of Section V shows what tightening α costs in certifiable savings, and the floor below shows where tightening it becomes impossible.

f: Assumptions.

Assumption 1 (Pinned snapshot). The model $\mathcal{M}$ and the decode parameters θ are fixed during certification. Oracle draws are distributed as $f_{V'}(r_i)$ throughout the certification window and are independent *across rows*. Provider-side batching that correlates concurrent responses [8], [9] would violate this; our runner issues independent requests, and Section V, "Independence across rows," compares sequential with concurrent requests. The validity proof uses independence only through non-interference: a sampled cell's draw must not depend on which other cells were sampled or in flight. Rows with identical bound content share one cell by design (Section IV), and hence one draw; that sharing is fixed by row content before sampling and leaves non-interference intact. Certificates are scoped to the snapshot the oracle draws came from; the cache may be older. A change of the served model ends the snapshot, and Section V, "Deployment scale" (the September run), measures such a change (change-of-world maintenance is complementary work).

Assumption 2 (Frozen stratification). The stratifier is a deterministic function of the cache, the edit, and row content, fixed before any oracle call. Any such stratifier preserves validity; its quality affects only power (Section V, "The stratification hierarchy").

Assumption 3 (Uniform within-stratum sampling). Sampled rows are drawn by a seeded uniform shuffle within each stratum, without replacement, and each sampled row receives one fresh oracle draw, independent across rows (Assumption 1). The sampling itself is fixed by the seeded canonical shuffle of Section IV. The calibration study of Section V checks that the implemented test holds the level Theorem 1 states, on planted strata whose rates sit just above the budget and on the fixed population that is least favorable to it.

g: No monotonicity shortcuts.

Folk reasoning suggests one-sided safety: a scope-widening edit should preserve TRUE cells, so cached TRUEs could be reused for free. Our measurements refute the unconditional form: under a widening edit, 45.7% of cached-TRUE cells flipped, driven largely by boundary instability, not semantics. Edit direction therefore enters sIVM only as a stratification hint; any directional certificate must be *conditional* on a measured violation rate, never assumed.

h: The impossibility floor is per-stratum.

Let s_j be stratum j 's self-flip rate: the probability that a fresh draw of a cell under the *unedited* prompt disagrees with

the cached draw, *among the cells whose cached draw put them in stratum j* . For the identity edit $\pi' = \pi$, p_j equals s_j by definition. Hence no valid procedure whose oracle is a single fresh draw can certify stratum j at $\alpha < s_j$, except with probability at most δ plus the chance that the realized flip count is at or below αn_j , which vanishes as n_j grows. Under the unedited prompt the certifiable budget is floored by model stability. Two scoping notes apply. First, the conditioning is not incidental. Because membership is decided by one draw, a minority-value stratum is selected partly *for* having flipped that way, so s_j is a property of the (cache, stratifier) pair, not of the column. Building the cache by a majority vote over several draws at write time lowers it (Section V, “Minority-stratum anatomy”). Second, the floor is a bound for the identity edit only. An edit that sharpens the output distribution, or shifts it toward a stratum’s cached value, can drive p_j below s_j , and the family study contains such strata (Section V, “What transfers”). For every other edit the floor is a reference point, not a bound.

The floors are unequal, and the column-level average describes neither stratum. We measure each floor as defined, the stored cached value against three fresh draws of the same prompt per row. On our Boolean column the cached-FALSE stratum has $s=2.6\%$ (95% row-cluster bootstrap [2.0, 3.1]) while the cached-TRUE stratum has $s=23.0\%$ ([18.6, 27.4]), against 4.5% pooled. Rows are bucketed by the stored cached value the certifier itself stratifies on, and the interval resamples rows, since draws from one row are dependent. The column-level 5.0% quoted elsewhere is the rate between pairs of fresh draws. Three otherwise separate observations follow from this single measurement. The FALSE stratum is the only one that certifies at all, and it is certifiable in principle down to budgets near its own floor. At $\alpha=0.05$ it certifies in 74.4% of replications, some only at the deepest look of the schedule, for 38.6% mean savings (Table 2). There, power rather than the floor is what binds. The TRUE stratum is never certified at any budget we tried: its floor exceeds every budget we ran. Its size sets the deployment-scale savings ceiling: with 7,715 of 89,184 cells cached TRUE, reuse cannot exceed 91.3%. The ceiling belongs to *this* cache, and a cache built by majority vote at write time would hand the certifier a more stable stratum. And any method that reuses cells from the cached-TRUE stratum inherits that stratum’s flip rate no matter what aggregate target it was given, which is what we observe when we run one (Section V, B2’). Column-level stability is the wrong unit; the certifier’s stratification recovers the right one, and a standing refusal on a stratum is a stability diagnosis of that slice of the column.

III. THE sIVM PROCEDURE

Given the edit $\pi \rightarrow \pi'$ and cached cells for the affected rows (Fig. 1):

- 1) **Diff.** Compute a word-level longest-common-subsequence (LCS) diff of the prompts; collect content terms of added/removed spans (the *delta*). The delta is recorded in the certificate for provenance and

consumed only by the optional refinement in step 2; it never enters the reuse decision itself.

- 2) **Stratify (frozen).** Partition rows by cached value and freeze the partition before any oracle call (Assumption 2). With a binary proxy this step is the stratification of proxy-certified selection [10], transported from one-shot selection to maintenance. What changes is the guarantee it carries: a bound per stratum instead of one aggregate bound over the selected set (Section V, B2’, isolates exactly that difference). Optionally refine each value stratum by an *interaction score*, a measure of how strongly the delta matches the row (lexical delta-term overlap, or the embedding cosine between the delta text and the row’s bound content), bucketed into tertiles. This refinement leaves validity intact (Assumption 2). It pays only when within-value flip risk is heterogeneous, and otherwise costs power through the δ/K split (Section V).
- 3) **Schedule.** For stratum j of size n_j , fix a doubling look schedule $L_j = (45, 90, 180, \dots)$ capped at n_j , with at most six looks. Split the stratum’s failure probability $\delta_j = \delta/K$ evenly across looks. Every experiment here pins $n_0=45$, at most six looks, and $\delta=0.1$. Here K is the number of cached values: 2 on the Boolean columns and 4 on the select column. In the free-text pilot of Section VIII there are 8 category strata. A look on a large Boolean stratum is therefore tested at the per-look level $\delta' = \delta_j/|L_j| = 0.1/12$, and a look on a large select stratum at $0.1/24$. A small stratum has fewer looks and a correspondingly larger per-look level.
- 4) **Sample and test.** Order the stratum’s rows canonically, shuffle them once with the seed (Assumption 3), and at each look t reveal one fresh draw for each row of the prefix (Assumption 1). Compute the exact finite-population upper confidence bound $u_{j,t}$ of Eq. (1) on the stratum flip rate at level $\delta_j/|L_j|$; *certify and stop* if $u_{j,t} \leq \alpha$; *stop for futility* if the empirical rate alone exceeds α .
- 5) **Apply.** Reuse all unsampled rows of certified strata (recording certificate provenance per cell); recompute the rest. Sampled rows are already fresh.

Algorithm 1 states the procedure in full.

a: The bound.

Fix a stratum of N cells ($N = n_j$), and give every cell one hypothetical fresh draw under V' , the coupling on which the presented-cells estimand is defined. Let M be the number of cells whose fresh draw flips. The sample of n cells is drawn uniformly without replacement and independently of those draws, and shows k flips. Conditional on the N outcomes, k is Hypergeometric(N, M, n), and $\Pr[\text{Hyp}(N, M, n) \leq k]$ is non-increasing in M . The exact upper confidence bound at level δ' is

$$u = \frac{1}{N} \max\{M : \Pr[\text{Hyp}(N, M, n) \leq k] > \delta'\}, \quad (1)$$

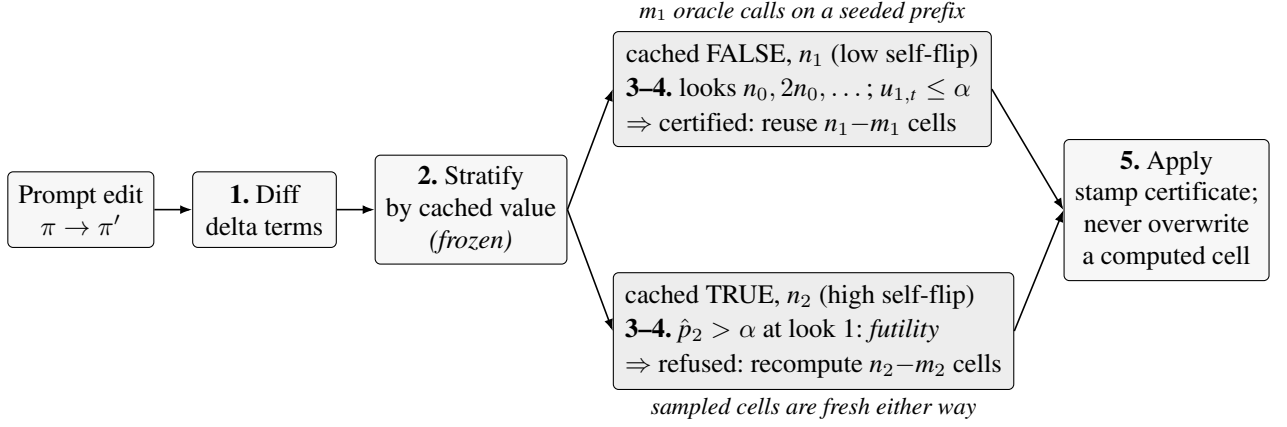


FIGURE 1. The procedure on the configuration our experiments pin. Stratification is frozen before any oracle call, so it moves power but never validity (Assumption 2); only steps 3–4 spend model calls, and they stop on evidence about a stratum’s *rate*, so their cost is bounded independently of the stratum’s size. The two lanes are the outcome on the primary model’s Boolean column: the stable stratum certifies and is reused under a certificate, while the volatile one is refused at every budget we tried and recomputed. A refusal is a stability diagnosis of that slice of the column (Section II), reached at the cost of one look instead of a full recomputation.

Algorithm 1 sIVM certification of an edit $\pi \rightarrow \pi'$

Require: cached cells $\{(r_i, c_i)\}$; edit $\pi \rightarrow \pi'$; budget α ; failure probability δ ; first look n_0 ; at most ℓ looks; mode $\in \{\text{default, strict}\}$; Assumptions 1–3

Ensure: reuse set $\mathcal{R}$, recompute set, certificate record

```

1:  $\Delta \leftarrow$  word-level LCS diff of  $\pi$  and  $\pi'$ 
2: strata  $1, \dots, K \leftarrow$  partition of the cells by cached value,
   optionally refined by an interaction bucket of  $\Delta$ ; freeze
3: for each stratum  $j$  with  $n_j$  cells do
4:    $\text{order}_j \leftarrow$  seeded shuffle of the stratum’s rows in
     canonical order;  $L_j \leftarrow (n_0, 2n_0, \dots)$  capped at  $n_j$ , at
     most  $\ell$  looks;  $\delta' \leftarrow \delta / (K |L_j|)$ 
5:   for each look  $n \in L_j$  do
6:     draw  $X'_i$  for the first  $n$  rows of  $\text{order}_j$  not yet drawn;
      $k \leftarrow |\{i \leq n : c_i \neq X'_i\}|$ 
7:      $u \leftarrow$  exact upper bound of Eq. (1) at  $(k, n, n_j, \delta')$ 
8:      $\alpha' \leftarrow \alpha$  in the default mode, else  $\alpha(n_j - n)/n_j$ 
9:     if  $n < n_j$  and  $u \leq \alpha'$  then
10:      certify stratum  $j$ ; break
11:    end if
12:    if  $k/n > \alpha$  then
13:      refuse stratum  $j$  (futility); break
14:    end if
15:  end for
16:  refuse stratum  $j$  if its schedule is exhausted
17: end for
18:  $\mathcal{R} \leftarrow$  unsampled cells of certified strata; recompute set
    $\leftarrow$  unsampled cells of refused strata
19: persist the certificate (budget,  $\Delta$ , per-stratum evidence)
   and apply  $\mathcal{R}$  additively
  
```

the finite-population form of the Clopper–Pearson bound [11], to which it tends as N grows. Because the coverage holds conditionally on every population realization, it holds unconditionally over sampling and draw randomness

together, and it bounds the realized flip count M rather than its expectation. This is the natural bound for a binary loss under the sampling the procedure performs. On a clean sample its cost is $\ln(1/\delta')/n$ to first order. The empirical-Bernstein bounds pay range terms with constants of $7/3$ (Maurer–Pontil, on $\ln(2/\delta')$) and $7/3 + 3/\sqrt{2}$ (Bardenet–Maillard, on $\ln(5/\delta')$) that a binary loss does not need. On a clean sample of $n=90$ from a stratum of 1,800 cells at the pinned per-look level, the exact bound reads 0.050. In the 81,469-cell deployment stratum it reads 0.052; there it coincides with the binomial limit to three digits. On the clean sample of 90 from 1,800 cells, the Maurer–Pontil bound [12] that the preprint of this article certified with (see the Availability section) reads 0.144, and the Bardenet–Maillard without-replacement bound [13] reads 0.317. Section V measures the difference on every configuration and prices a betting confidence sequence [14]; time-uniform boundaries [15] are discussed there as the remaining lever. The system exposes the alternatives as ablation arms.

Theorem 1 (Validity under peeking). Let K be the number of strata produced by the frozen stratifier, $\delta_j = \delta/K$, and let every look use the exact bound of Eq. (1) at level $\delta_j/|L_j|$. Under Assumptions 1–3, with probability at least $1 - \delta$ over sampling and oracle draws, every certified stratum j has realized whole-stratum flip count $M_j \leq \alpha n_j$; consequently $\text{FR}_j^{\text{pres}} \leq \alpha$ for every certified stratum and for every union of certified strata (the empty certification is vacuously valid). In the strict (reuse-set) mode, where certification at sample size m_j requires $u_{j,t} \leq \alpha(n_j - m_j)/n_j$, the same argument gives $M_j \leq \alpha(n_j - m_j)$, hence $\text{FR}_j^{\text{reuse}} \leq \alpha$. The statements hold for any per-look bound that is a valid upper confidence bound at that level for uniform without-replacement draws, including the Bardenet–Maillard bound [13]; the Maurer–Pontil bound [12] is stated for independent draws and is retained only as an ablation arm.

Proof. Fix stratum j and condition on its n_j fresh-draw outcomes, of which M_j flip. If $M_j > \alpha n_j$, certifying at look t requires $u_{j,t} \leq \alpha < M_j/n_j$, i.e., the observed count to fall in the lower tail that Eq. (1) rejects at level $\delta_j/|L_j|$; by the monotonicity of the hypergeometric tail in M , that event has probability at most $\delta_j/|L_j|$ over the sampling, for every population realization, hence unconditionally. Because the looks are prefixes of one shuffle fixed before sampling, the prefix of size n_t (the t -th entry of L_j) is, marginally, a uniform without-replacement sample of that size, whether or not the procedure reaches look t . The bound therefore fails at any one scheduled look with probability at most $\delta_j/|L_j|$, and a union over all scheduled looks covers whichever look the data stops at. That union bound over the $|L_j|$ looks and the K strata gives δ ; this is the Bonferroni instance of Learn then Test [16], each (stratum, look) pair testing the null $M_j > \alpha n_j$ with a certificate as the rejection. The futility rule only stops sampling, never certifies, so it cannot increase the error probability. On the good event, $\text{FR}_j^{\text{pres}} = (M_j - k_j)/n_j \leq \alpha$, where k_j is the number of flips observed in the sample, and any size-weighted union of certified strata inherits the bound. In the strict mode the good event is $M_j \leq \alpha(n_j - m_j)$; at worst every flip falls in the remainder, so $\text{FR}_j^{\text{reuse}} \leq \alpha$. The conditioning uses Assumption 1 only through non-interference: the population of fresh-draw outcomes must be the same whichever cells the shuffle selects. $\square$

Remark. The Bardenet–Maillard bound is proven for the same uniform without-replacement draws (its Serfling factor shrinks the variance term as the sampled fraction grows; its constants are those of Theorem 4.3 of [13]), so the argument transfers to it unchanged. The Maurer–Pontil bound is stated for independent draws. Whether it remains valid at the sampled fractions our runs reach is not something we prove, which is why the pinned bound is the exact one. Those fractions are large: the schedule’s deepest look draws 1,440 cells of a stratum, most of a benchmark stratum, and the certifier samples up to 74% of a benchmark table.

b: Power.

In a large stratum a clean sample certifies once $(1 - \alpha)^n \leq \delta'$, i.e., at $n^* = \lceil \ln \delta' / \ln(1 - \alpha) \rceil$. Under the pinned procedure on a Boolean column ($K=2$, six looks), this gives $n^*=94$ at $\alpha=0.05$, against 257 for the Maurer–Pontil bound. The doubling schedule cannot stop on n^* : the first look at or above it is the one it takes. A clean stratum of $n_j=5,000$ therefore pays 180 draws in calibration. A clean stratum of $n_j=600$ pays 90, because its finite-population correction and its shorter schedule’s larger per-look level bring the earlier look under α . Either way, savings require strata much larger than n^* , i.e., large tables, which is precisely the deployment regime. Near the frontier, certifying a stratum with true rate p requires roughly $n \gtrsim z_{\delta'}^2 p(1-p)/(\alpha-p)^2$ samples, where $z_{\delta'}$ is the standard normal quantile at level δ' (the normal approximation to the exact test). This explains the measured collapse of savings as $\alpha \downarrow p$. On SO formatting (Table 2),

mean savings are 88.1%, 82.6%, and 38.6% at $\alpha=0.2$, 0.1, and 0.05, with refusal at 0.02.

IV. IMPLEMENTATION

sIVM runs inside *lore*, a working AI-table system. Its natural-language search bar compiles to a versioned filter specification executed identically on MySQL and Elasticsearch (conformance-tested for count, page, sort, aggregation, and hydration parity on 89,184 rows). AI columns are per-cell jobs with typed structured outputs, bounded concurrency, retry, and a content-hash cache keyed on *only the bound fields* $\beta(\pi)$. Unrelated row changes therefore never invalidate the cache, and content-duplicate rows share one computation. Every run passes a pre-run cost gate and lands in a ledger; prompt edits create immutable versions (the sIVM substrate); certificates persist with full strata evidence, and reused cells carry visible provenance in the UI. Decode is pinned ($T=0$) for all cell computation per Assumption 1.

a: What the user sees.

In the table view a reused cell carries a shield mark beside its value. Its tooltip shows the cell’s stored rationale with the note that it was reused from the previous version under a statistical certificate. A cache hit is annotated as one in the same place. Certifying a column version returns the certificate’s summary to the user: the rows in scope, α , the cells sampled, reused, and to be recomputed, and the flips observed in the sample. A refused stratum has no mark of its own: its cells are recomputed and served as ordinary fresh cells, and the refusal is visible in the certificate record. Whether this surface changes how a person acts on the column was not studied (Section VIII).

b: Storage model.

Cells are keyed by (column, row, prompt version), so versions of a column *coexist* instead of replacing one another, and a secondary index on (column, version, model, content hash) serves the cache. This is the one design decision sIVM depends on. Systems that key a cell by row alone, or that hash the prompt into the cache key, destroy the old materialization at the moment of the edit. The question this article asks cannot then be posed: there is nothing left to certify. Keeping versions side by side turns “which cached cells still transfer?” into a join between two versions of one table, which is why the certifier is a query over the store, not a service beside it. Two details of that query matter for reproducibility and for Assumption 1. The certifier orders each stratum’s rows canonically (by content hash, then row identifier) before the seeded shuffle, so the sample path is a function of the seed and the data alone. An earlier version of the certifier took the order the storage engine happened to return. That order changed when the query planner switched indexes, and the same seed then drew a different sample (Section V). And a model snapshot is a version of its own: oracle draws taken after a change of the served model are stored under a new prompt version carrying the same template. Draws from two

snapshots therefore never share a key and cannot be mixed in one certification path.

c: Certificates are records, not decisions.

A certificate persists the budget α , the prompt delta, the per-stratum evidence (size, cells sampled, bound attained, certified or refused), and the counts of sampled, reused, and recomputed cells. It is therefore auditable after the fact: the realized-error audits in Section V are recomputed, from the row identifiers each run persisted and the stored cells, by a script that imports nothing from the certifier. Applying a certificate is a single `INSERT . . . SELECT` that carries certified values forward under the new version and stamps the certificate identifier on each one.

That insert carries an anti-join against the target version, so reuse is strictly *additive*: a certificate can supply a value where none was computed, and can never overwrite one that was. An earlier implementation used an upsert, which is the obvious way to write it and is silently destructive. Applying a certificate after a partial recompute reverted freshly computed cells to the stale values the certificate had been asked to reuse. The bug is invisible in aggregate error, because a reverted cell and a reused cell are the same row with the same value. It surfaces only if something outside the certifier knows what that cell should have said. Any system that maintains stochastic cells across versions has this hazard, and the guard belongs in the write path, not in code review.

V. EXPERIMENTS

a: Workloads.

All measurements use two public corpora and three prompt-defined columns (Section VI gives the exact edits, and the Availability section the sources). On the 2023 SO Developer Survey (89,184 respondent rows) a Boolean column asks whether the respondent is a data-platform specialist, judged from structured facets of the row (role, databases, platforms, tools). The column exists twice, as the production column and as a lab replica seeded from the production column's second version, so that edits can be studied without touching the deployed table. On Djinni English candidate profiles (210,250 rows) a four-way ordinal select column assigns a seniority tier from the position, years of experience, and free-text profile. Five edit pairs across the three columns cover formatting-only edits, synonym rewording, scope widening, and a criteria change; each is labeled on a seeded $n=2,000$ evaluation vector of its corpus.

b: Scope.

One budget-class reasoning model carries the full experimental program. A further 6 model families from four other providers, three of them successive Gemini generations, replicate the lab column's two pairs; 5 of them replicate all five pairs. The quantitative frontier reported below is specific to the (edit, model) pair while the qualitative structure is what the family study tests (Section VIII). Table 3 lists every model identifier and run date. The gateway exposes no snapshot

identifier finer than these names (the primary model's carries a date suffix; the others do not), so the run dates are the snapshot record. The September re-certification below shows what one identifier serves changing between run dates.

c: Models and decoding.

Every cell in the primary program uses DeepSeek V4 Flash, called through the Vercel AI Gateway (gateway identifier `deepseek/deepseek-v4-flash-0731`). Those calls were made between 4 August 2026 and 5 August 2026, and again on 29 September 2026 for the re-certification and the assumption checks (and on 30 September 2026 for the larger concurrent arm of the independence check). The model-family study calls further families through the same gateway from 15 August 2026 to 29 September 2026. Every call passes temperature 0 and no other decode parameter. Maximum output length and reasoning effort stay at the provider's defaults. No decode seed is sent, since the seeds in this article govern row sampling: `mulberry32`, main seed 42, sampling replications 1000 to 1999. Cells are requested as typed structured outputs against a JSON schema through the gateway's client library. A call whose output fails the schema or whose request errors is retried by the client. A cell that still fails is stored with an error status and excluded from labels and from every rate in this article. No such cell exists among the cells written through the column runner; the probe scripts that call the model directly report their own exclusions where they occur. The embedding proxy in baselines B1 and B3 and in the interaction stratifier is `openai/text-embedding-3-small`, also through the gateway. Every ground-truth pass, sample, and replay is seeded and reproducible from released scripts.

d: Labels and spend.

Flip labels come from single fresh draws under the new version, which matches the estimand (a label answers "would a recompute differ from the cache now?"). A three-draw relabeling of the headline pair quantifies the instrument's own variance. With fresh draws on both sides the edit flips 7.2% of cells, and 6.30% under majority-vote labels (126 of the 2,000 rows). The stored single-draw labels flip 127 rows (6.35%), which Table 2 prints as 6.3%. They flipped 120 rows (6.0%) when the relabeling ran. Afterwards, on 5 August 2026, the earlier apply step wrote certificate copies over the 1,811 of the vector's `v2` cells that the certificate reused (the fault Section IV describes), and those cells were drawn again. Table 2 uses the cells as they stand. Between the earlier labeling and the majority vote, 4.8% of rows relabel. These are draws of one population quantity on the same 2,000 rows, and the spread between them is what single-draw labeling costs. We report the spread instead of picking whichever instrument flatters the result. (The within-version floor, 5.0%, is the separate quantity that limits certifiability under an unedited prompt.) Table 4 reconciles every flip-rate figure quoted for the headline pair and its cached-TRUE stratum with the edit, the cells, and the instrument behind it. Labels are individually

TABLE 2. The pinned primary procedure (cached-value strata; exact bound; adaptive doubling looks $n_0=45$, six looks; $\delta=0.1$; presented-cells estimand) on all five edit pairs, $n=2,000$ each at temperature 0. Columns: *flip*, the pair’s label flip rate; *oracle* and *reused*, the calls spent and the cells reused by the main seed; FR_{pres} and FR_{reuse} , the main seed’s realized false-reuse over presented cells and over the reuse set alone (the latter with a 95% Wilson confidence interval, CI); *savings*, the mean over $B=1,000$ sampling replications on frozen ground-truth labels with a 95% percentile interval (PI); *cert.*, the share of those replications that issued any certificate; *exceed.*, the share of the certifying replications (not of B) whose realized reuse-set rate exceeded α , a metric the default mode does not certify (Section II). The table is generated mechanically from the result files; earlier exploratory sweeps with other stratifiers and schedules are reported as ablations, not headline numbers.

Edit	α	flip	oracle	reused	FR_{pres}	FR_{reuse} [95% CI]	savings % [95% PI]	cert.	exceed.
SO formatting-only	0.02	6.3%	225	0	–	–	0.0 [0.0,0.0]	0.0%	–
SO formatting-only	0.05	6.3%	1,485	374	0.55%	2.67 [1.5,4.9]	38.6 [0.0,81.7]	74.4%	0.0%
SO formatting-only	0.10	6.3%	90	1,769	3.20%	3.28 [2.5,4.2]	82.6 [72.7,88.4]	98.9%	0.0%
SO formatting-only	0.20	6.3%	90	1,769	3.20%	3.28 [2.5,4.2]	88.1 [86.2,88.4]	100.0%	0.0%
SO synonym rewording	0.02	8.9%	90	0	–	–	0.0 [0.0,0.0]	0.0%	–
SO synonym rewording	0.05	8.9%	90	0	–	–	0.0 [0.0,0.0]	0.0%	–
SO synonym rewording	0.10	8.9%	90	0	–	–	55.6 [0.0,89.0]	81.4%	0.0%
SO synonym rewording	0.20	8.9%	225	1,645	5.81%	6.44 [5.4,7.7]	87.4 [82.3,89.0]	99.9%	0.0%
SO scope widening	0.02	9.4%	90	0	–	–	0.0 [0.0,0.0]	0.0%	–
SO scope widening	0.05	9.4%	765	0	–	–	1.0 [0.0,0.0]	1.2%	100.0%
SO scope widening	0.10	9.4%	405	1,441	4.50%	5.62 [4.5,6.9]	66.3 [0.0,87.8]	88.8%	0.0%
SO scope widening	0.20	9.4%	90	1,756	5.33%	5.47 [4.5,6.6]	86.6 [85.5,87.8]	99.9%	0.0%
Djinni formatting-only	0.02	12.3%	180	0	–	–	0.0 [0.0,0.0]	0.0%	–
Djinni formatting-only	0.05	12.3%	315	0	–	–	0.0 [0.0,0.0]	0.0%	–
Djinni formatting-only	0.10	12.3%	495	383	4.17%	8.09 [5.8,11.3]	8.4 [0.0,32.6]	73.8%	8.4%
Djinni formatting-only	0.20	12.3%	953	1,047	8.33%	10.12 [8.4,12.1]	52.1 [32.6,73.6]	100.0%	0.0%
Djinni criteria change	0.02	12.4%	180	0	–	–	0.0 [0.0,0.0]	0.0%	–
Djinni criteria change	0.05	12.4%	315	0	–	–	0.0 [0.0,0.0]	0.0%	–
Djinni criteria change	0.10	12.4%	540	378	3.52%	6.88 [4.7,9.9]	14.4 [0.0,32.4]	82.8%	5.4%
Djinni criteria change	0.20	12.4%	745	1,255	7.50%	11.00 [9.4,12.8]	48.6 [32.4,69.5]	100.0%	0.0%

noisy with about a point of draw-to-draw spread in the aggregate, which the replication intervals of Table 2 dominate. It is population functionals, not labels, that the guarantee addresses. This is the same picture the LLM-as-annotator literature draws, where model labels reach crowd-worker or expert agreement on many tasks but with task-specific noise that has to be validated rather than assumed [17]–[20]. Ledgered model spend across every experiment in this article is \$12.01 for 100,683 billed cells, read from the system’s own cost ledger. That figure is a lower bound, not a total. The ledger records only cells written through the column runner. A further 146,753 cells were computed by probe scripts that call the model directly (the self-flip probes, the majority-vote study, the multi-draw relabeling, the model-family runs, the independence check, and the free-text pilot’s draws).

e: Baselines.

Four comparisons isolate what sIVM adds. (B1) *Verified-cache similarity has no cross-version signal*: the per-row similarity between the v1-bound and v2-bound prompts, the quantity a verified semantic cache thresholds, predicts cell flips at chance as measured by the area under the receiver-operating-characteristic curve (AUC 0.454 formatting, 0.499 widening). On the formatting edit the score’s entire range is 0.96–0.99, because shared row text dominates. Adding “a version key” to such a cache therefore cannot produce a sound cross-version policy. Our interaction score carries real signal exactly when the edit is semantic (AUC 0.669 on widening) and none above chance when flips are noise-driven (AUC 0.406 on formatting). That is why the cached-value dimension of stratification carries the formatting case.

(B2) *Aggregate-only certification (no strata)*, run with the same schedule and budgets as the pinned procedure, is valid for its aggregate estimand and silently corrupts subgroups. On the formatting edit it certifies at $\alpha=0.1$ (82.0% savings, 6.46% aggregate error) while the reused cached-TRUE subgroup carries 35.3% error. At $\alpha=0.2$ on the same edit it certifies 97.8% at 6.39% aggregate with 37.2% subgroup error. On the widening edit it refuses at $\alpha=0.1$ and certifies 95.5% at $\alpha=0.2$ with 44.6% subgroup error. Per-stratum bounds are what make the guarantee mean something for every user-visible slice of the table, and the effect is not an artifact of giving the baseline a shorter schedule.

(B2') *A SUPG-style competitor, matched*. The closest existing method is proxy-certified selection [10]: score rows with a cheap proxy, spend an oracle budget labeling a sample, threshold the proxy so the selected set’s error clears the target. We transported it to maintenance with the proxy the reduction in Section III names, the cached value itself. We matched it to sIVM on every axis we could: the same number of oracle calls, identical labels, identical error target, the same exact bound on the selected population, the same total failure probability δ , and the same reported quantity. Each method’s δ is split across the number of tests its own procedure performs: sIVM across the Boolean column’s two strata and six looks, $\delta/12$ per test; the competitor across its candidate thresholds, $\delta/3$. The reported quantity is the error among reused cells, not the presented-cells rate sIVM certifies.

Matched this way it is a live competitor that wins on volume and loses on where its errors land. At $\alpha=0.2$ on the formatting edit it reused 1,910 cells to sIVM’s 1,769 from the same 90 labels. At 6.54% realized error it satisfies the bound it

TABLE 3. Every model the article uses, by gateway identifier, with the gateway's list prices as fetched on 29 September 2026.

Gateway identifier	Role	Run dates	\$/M tokens in / out
deepseek/deepseek-v4-flash-0731	primary model: every cell of the main program, the deployment run, the drift and independence checks, the free-text pilot's cells and its judge	4 August 2026 to 5 August 2026; 29 September 2026; 30 September 2026	0.08 / 0.15
openai/text-embedding-3-small	embedding proxy (baselines B1, B3; interaction stratifier)	with the main program	0.02 / 0.00
google/gemini-2.5-flash-lite	family study, all five pairs	15 August 2026; 29 September 2026	0.10 / 0.40
google/gemini-3-flash	family study, lab column's two pairs	15 August 2026	0.50 / 3.00
zai/glm-4.7-flash	family study, all five pairs	15 August 2026; 29 September 2026	0.07 / 0.40
alibaba/qwen3.7-flash	family study, all five pairs	15 August 2026; 29 September 2026	0.03 / 0.13
openai/gpt-5-nano	family study, all five pairs	29 September 2026	0.05 / 0.40
google/gemini-3.8-flash	family study, all five pairs	29 September 2026	0.75 / 3.75
meta/muse-spark-1.3-contributor	free-text pilot: second labeler of the calibration pairs, reported beside the hand labels	30 September 2026	0.10 / 0.20

TABLE 4. Reconciliation of the flip-rate figures quoted in this article. Each is a different quantity, a different instrument on the same quantity, or the same instrument on an earlier draw; none is a restatement of another.

Rate	Edit	Cells	Instrument (artifact)
<i>Headline pair: SO formatting-only, Boolean lab column</i>			
6.3%	formatting v1→v2	$n=2,000$ evaluation vector	stored v2 oracle cells against the stored v1 cache, one draw each side (127 flips, 6.35% to two decimals); the Table 2 label instrument (exp8)
6.0%	formatting v1→v2	same rows	the same comparison on the v2 cells as first drawn (120 flips), recorded before 1,811 of them were re-drawn on 5 August 2026 (exp10)
6.30%	formatting v1→v2	same rows	majority of three draws on each side (126 flips; exp10)
7.2%	formatting v1→v2	same rows	fresh single draws on both sides, neither the cache (exp10)
5.0%	identity (no edit)	same rows	pairs of fresh draws of one version: the column-level self-flip rate (exp10)
2.6%, 23.0%	identity	cached-FALSE, cached-TRUE rows	the stratum floors: the stored v1 value against each of three fresh v1 draws (4.5% pooled; exp10)
<i>Cached-TRUE stratum figures</i>			
37.1%	formatting v1→v2	cached-TRUE rows of the vector	single fresh draw against the cache (exp4)
24.7%	formatting v1→v2	same	vote-of-3 on both sides (exp4)
45.7%	widening v1→v2, production column	199 cached-TRUE rows	single fresh draw against the cache (Wilson [39.0, 52.7])
35.3%	formatting v1→v2	cached-TRUE cells B2 reused at $\alpha=0.1$	realized error among the cells the aggregate-only baseline reused, an adversely selected subset, not a population rate (exp7)
37.2%	formatting v1→v2	same at $\alpha=0.2$	same (exp7)
44.6%	widening v1→v2, production column	cached-TRUE cells B2 reused at $\alpha=0.2$	same (exp7)

was given, against sIVM's 3.28%. On the widening edit the same pattern holds: 1,910 reuses at 9.58% versus 1,756 at 5.47%. The extra volume is drawn from the volatile stratum, 183 cached-TRUE cells on the formatting edit and 196 on the widening edit. That stratum's floor on the lab column is 23.0%, and no aggregate target can make reuse safe. sIVM reuses none of those cells. Section II predicts this behavior: an aggregate target is satisfied by any mixture that averages out, so it buys volume with the correctness of the least stable subgroup. Per-stratum bounds are what extend the guarantee

to that part of the table.

With the cached value as the proxy there are only three candidate thresholds, and at $\alpha=0.2$ the aggregate bound clears at the loosest one. The arm therefore selects 100.0% of the rows it was allowed to on the formatting edit and 100.0% on the widening edit. Matched this way, the competitor reduces to B3, reuse everything, with a certificate attached: its bound is satisfied, its aggregate error is real, and its behavior is indistinguishable from the guarantee-free status quo. The difference sIVM makes here is that per-stratum bounds refuse

a subgroup an aggregate target is willing to spend.

We note two caveats on the same experiment. First, the cached value is the proxy the reduction names, and the result does not depend on that choice. Run instead with an embedding-interaction proxy, and with its much larger threshold search corrected accordingly, the competitor certifies in 3 of the 4 (edit, budget) configurations and lands in the same place. At $\alpha=0.2$ on the formatting edit it reuses 1,899 cells at 6.53% error, 180 of them cached-TRUE cells that flip at 37.2%. On the widening edit it reuses 1,856 at 9.21%, 168 of them cached-TRUE cells at 50.6%. The proxy changes how much the competitor reuses, not where its errors land. Second, with the cached-value proxy the competitor's per-test level is looser than sIVM's, because it runs fewer tests. The embedding arm, tested at a stricter level than sIVM's, still reuses more than sIVM at $\alpha=0.2$, so that advantage does not rest on the looser level. The comparison is matched on the quantity a user sets (δ in total), not on the level of any individual test. We report both levels ($\delta/12$ and $\delta/3$) so the reader can see which one the result rests on.

(B3) *Guarantee-free embedding transfer* (GPTCache-style [21], $\tau=0.95$) reuses every cell on the formatting edit and 84.0% of them on the widening edit, silently absorbing error at about the population flip rates. The cells it reuses flip at 6.3% on the formatting edit and at 9.8% on the widening edit, where the population rate is 9.4%. This is the deployed status quo. That τ sits on the shoulder of the widening edit's collapse, quantified in the threshold sweep below. Table 5 lines these policies up beside the two trivial ones, always recompute and always reuse, on the SO formatting and widening edits at $\alpha=0.2$. The trivial policies bracket the design space (zero error at full cost; full savings at the population flip rate). The aggregate methods buy volume from the unstable subgroup, and sIVM is the only row whose reused cells carry a per-stratum bound.

f: Noise floors and column stability.

Replicate self-runs give the two column types' intrinsic self-flip rates. The Boolean column's floor is 5.0%, measured from within-version draw pairs over $n=2,000$ rows (an initial $n=200$ two-draw probe read 3.0% and is superseded). The four-way select's floor is 6.5% ($n=200$ probe, a smaller sample than the Boolean floor's and stated as such). Every observed self-flip was between adjacent tiers. At default temperature the Boolean floor rises to 9.5% on that same $n=200$ probe, whose $T=0$ arm read 3.0%. The temperature comparison rests on the within-instrument contrast, and the default-temperature floor alone exceeds a 5% budget, which motivates decode pinning. Formatting-only edit flip rates sit above each floor (6.3% over 5.0%; 12.3% over 6.5%). On the Boolean column, formatting flips are floor-dominated. On these two columns the flip rate tracks the column's own floor more closely than the edit's semantic class (the Djinni criteria change lands at 12.4%, indistinguishable from its formatting edit at 12.3%). Five author-written pairs cannot say how general that is.

g: Other model families.

Every measurement above uses one model, so we ran the lab column's two pairs on 6 further budget-class families from other providers. The family runs use the same seeded rows as the primary evaluation vector, the same prompts, $T=0$, one fresh draw per cell, and the pinned procedure (Table 6). Each family's cache is its own draw under the old version and its floor a second draw of that version. A flip is therefore the family disagreeing with itself across the edit, never a comparison with the primary model. Three properties transfer across families; one does not.

h: What transfers.

First, every family has a measurable self-flip floor, from 0.4% to 34.8% across them, and an edit whose flips are floor-dominated is not certifiable below it (Section II). GPT-5 nano, whose pooled floor of 34.8% sits above every budget in the grid, flips 39.6% of cells under the formatting edit, barely above that floor. It is refused on every stratum of that edit after 90 oracle calls. The floor is not a bound for edits in general, and two strata show it. Under the formatting edit GLM-4.7-Flash's cached-FALSE stratum flips 17.0% against an identity floor of 24.4%, and GPT-5 nano's cached-TRUE stratum flips 24.3% against 44.9%. Second, certified reuse is not a property of the primary model. On Gemini 3 Flash (floor 0.4%, formatting flip 4.5%) the formatting edit certifies the cached-FALSE stratum at both budgets. On Qwen3.7-Flash (floor 5.5%, flip 14.3%) it certifies both strata at the loose budget after most of the schedule. On the three families whose formatting edit is not benign, the synonym pair certifies a stratum instead. It is the cached-FALSE stratum on Gemini 2.5 Flash-Lite, and the cached-TRUE stratum on GLM-4.7-Flash and on GPT-5 nano, where it is the one stratum the loose budget admits. On GPT-5 nano that certificate sits below the family's pooled floor. The two are compatible: the floor is a per-stratum quantity, it was measured under the first version's prompt, and it binds for the identity edit only. Gemini 3.8 Flash, the sixth family, certifies both strata at both budgets under the formatting edit. That edit flips 0.85% of the family's cells (17 of 2,000), just under the family's floor of 0.95% (19 of 2,000). At least one stratum certifies on 6 of the 6 families, which is the sense in which certification reproduces. In every certified entry of Table 6 the realized presented-cells error is inside the budget. Third, refusals stay conservative: a refused stratum is never reused. Where a family refuses a stratum whose flip rate is below the budget (GLM-4.7-Flash's cached-FALSE stratum on the formatting edit, 17.0% against $\alpha=0.2$), the futility rule stopped at the first look because the sample rate exceeded the budget. That is a power miss the frontier of Fig. 2 predicts near the budget, not a validity failure.

i: Benign edits by family.

What does not transfer is which edits are benign. On the primary model the formatting-only edit is floor-dominated (6.3% over 5.0%). The same edit flips 25.4% of cells on Gemini 2.5 Flash-Lite (floor 4.9%) and 25.0% on GLM-4.7-Flash

TABLE 5. Reuse policies on the SO formatting and scope-widening edits at $\alpha=0.2$ ($n=2,000$, main seed). Realized error is the flip rate among reused cells; the trivial policies' errors follow by construction (always-recompute has none; always-reuse carries the population flip rate). B2 additionally reports the error of its reused cached-TRUE subgroup in parentheses. B2' is matched to the 90 labels sIVM spent; B2 runs the pinned schedule without strata and stopped after 45 labels on the formatting edit (360 at $\alpha=0.1$) and 90 on the widening edit.

Policy	Guarantee	Formatting		Widening	
		reused	error	reused	error
Always recompute	none needed (zero error)	0.0%	0.0%	0.0%	0.0%
Always reuse (status quo)	none	100%	6.3%	100%	9.4%
B3 GPTCache, $\tau=0.95$	none	100%	6.3%	84.0%	9.8%
B2 aggregate-only	aggregate	97.8%	6.39% (37.2%)	95.5%	9.37% (44.6%)
B2' SUPG-style, matched	aggregate	95.5%	6.54%	95.5%	9.58%
sIVM (this work)	per stratum	88.4%	3.28%	87.8%	5.47%

(floor 10.3%). It does so in both cached-value strata: cached-TRUE and cached-FALSE flip 28.7% and 24.6% on the former, 27.0% and 17.0% on the latter. When the edit is decomposed on Gemini 2.5 Flash-Lite, the two paragraph breaks alone flip 23.2% and the one-word case change alone 11.0%. The flips run one way (24.6% of them TRUE→FALSE on Gemini 2.5 Flash-Lite, 86.4% on GLM-4.7-Flash): a shift of the model's decision threshold, in opposite directions on the two families, not noise. On all three families whose formatting edit fails it is the synonym rewording that certifies a stratum, because its flips concentrate in one cached value and leave the other reusable. The label ordering that intuition suggests (formatting benign, semantic edits not) therefore inverts on some families and holds on others. Benignness is a property of the (edit, model) pair, and the certifier measures it per stratum without ever consulting the label. sIVM's value is a function of the model's self-consistency and of the edit's effect on that model, both of which it measures before reusing anything.

j: Beyond the lab column.

The remaining three pairs (the production column's scope widening and the two Djinni select pairs) were run on 5 of these families with the same protocol (Table 7). All five pairs are therefore complete on 5 of these families (Qwen3.7-Flash, Gemini 2.5 Flash-Lite, Gemini 3.8 Flash, GPT-5 nano, GLM-4.7-Flash). On those pairs, 5 of the 5 certify at least one stratum, and the same reading holds: which pair is benign, and on which stratum, is a property of the family. The select column's floor varies across families (Table 7) in the same way the Boolean column's does.

k: Main results.

Table 2 reads as follows. On the stable Boolean column the certifier reuses aggressively where it is safe. The formatting edit certifies at $\alpha=0.2$ with 88.1% mean savings (interval [86.2, 88.4]). The main seed's realized presented-cells error is 3.20% and its reuse-set error 3.28% (Wilson interval [2.5, 4.2]), the FR_{pres} and FR_{reuse} columns of the table. At $\alpha=0.1$ the same edit certifies with 82.6% mean savings; the main seed certifies at its first look, 90 oracle calls, and the replications span 90–405. At $\alpha=0.05$ it certifies in 74.4% of

replications. Semantic edits are refused at tight budgets. On the less stable select column the certifier certifies at $\alpha=0.2$, and at $\alpha=0.1$ in most replications; it treats the formatting and criteria edits alike, as their flip rates warrant. Of the 20 default-mode configurations, 12 certify in at least one replication. Budgets below a stratum's intrinsic rate are refused, and the refusals are the frontier of Fig. 2, not failures of the procedure. Realized presented-cells error is inside α in every main-seed certification.

l: Certificates that exceed the budget.

Across the 1,000 replications of every configuration, a certificate whose realized presented-cells error exceeds α occurs in 1 configuration (SO scope widening at $\alpha=0.05$, where the certified stratum's flip rate sits just above the budget). It certifies in 1.2% of replications. Because that stratum's realized flip rate sits above the budget, every one of those certificates is unsafe in the sense of Theorem 1: an unconditional rate of 1.2% against the per-stratum $\delta_j = \delta/K = 0.05$ of that column. In 75% of them the presented-cells rate also exceeds α , an unconditional 0.90%. That is an exact test spending its δ_j at a tight null (a stratum whose flip rate sits just above the budget), the behavior the calibration study below measures at scale, not a defect. The Maurer–Pontil bound of the preprint never produced such a certificate because it left almost all of δ unspent (Section V, “The bounds compared”). The reuse-set metric evaluated on default-mode certifications exceeds α in 3 frontier configurations: SO scope widening at $\alpha=0.05$; Djinni formatting-only at $\alpha=0.1$; Djinni criteria change at $\alpha=0.1$. The exceed. and cert. columns of Table 2 give the shares. This does not violate δ , which bounds the presented-cells event, nor Theorem 1, whose reuse-set guarantee applies only to certifications made in reuse-set mode.

m: The strict mode.

The strict reuse-set mode, whose deflated threshold shrinks with sampling depth, prices the stronger guarantee, and the price depends on where the stratum's flip rate sits relative to the budget. On the headline edit at $\alpha=0.2$ it is free: 88.1% mean savings against 88.1% in the default mode (88.4% on the main seed, as in the default mode), certifying in every replication. One budget tighter it costs two points of savings

TABLE 6. Model families on the lab column's two edit pairs under the pinned procedure (presented-cells estimand, $\delta=0.1$; the same seeded rows as the primary evaluation vector, one fresh draw per cell, $T=0$). Entries give certified savings with the realized presented-cells error in parentheses, or "refused" with the oracle calls spent before the futility stop. The primary row is the main seed of Table 2 (whose savings are replication means), with the column-level floor of the noise-floor paragraph. Here n is the rows requested; a few requests per family failed, leaving at least 1,966 usable rows per measurement. A family whose floor and flip agree to one decimal is printed to two. Gemini 3 Flash ran these two pairs only: in its reasoning regime the remaining pairs would have been the costliest runs of the study, and they were left out.

Family	n	floor	flip	formatting-only ($v1 \rightarrow v2$)		synonym rewording ($v2 \rightarrow v3$)		
				$\alpha=0.1$	$\alpha=0.2$	flip	$\alpha=0.1$	$\alpha=0.2$
DeepSeek V4 Flash (primary; main seed)	2,000	5.0%	6.3%	88.4% (3.20%)	88.4% (3.20%)	8.9%	refused (90)	82.3% (5.81%)
Gemini 2.5 Flash-Lite	2,000	4.9%	25.4%	refused (90)	refused (90)	31.5%	63.0% (0.84%)	63.0% (0.84%)
Gemini 3 Flash	2,000	0.4%	4.5%	76.5% (0.88%)	83.2% (1.05%)	6.2%	69.5% (1.91%)	85.6% (2.78%)
GLM-4.7-Flash	2,000	10.3%	25.0%	refused (90)	refused (90)	31.3%	refused (225)	57.3% (9.64%)
Qwen3.7-Flash	2,000	5.5%	14.3%	refused (765)	61.8% (8.35%)	27.2%	42.8% (3.67%)	49.5% (4.25%)
GPT-5 nano	2,000	34.8%	39.6%	refused (90)	refused (90)	39.7%	refused (90)	41.3% (12.30%)
Gemini 3.8 Flash	2,000	0.95%	0.85%	93.3% (0.65%)	95.5% (0.70%)	4.9%	90.5% (0.22%)	90.5% (0.22%)

TABLE 7. The remaining three edit pairs on further model families (same protocol as Table 6; floors are each family's self-flip rate on that column, measured in the same run; the primary model's select-column floor is the $n=200$ probe of the noise-floor paragraph). The primary row is the main seed of Table 2; its widening-column floor is not shown, because the primary model's self-flip rate under the production column's first version was not measured (the lab column's 5.0% is under the widened definition). Gemini 2.5 Flash-Lite holds 5 cached-TRUE cells on the widening column, fewer than the first look; that stratum is drawn in full, so its 5 draws count toward the family's calls on that pair and, with nothing left to reuse, it is never certified.

Family	SO scope widening (production column)				Djinni seniority tier (four-way select)							
	floor	flip	$\alpha=0.1$	$\alpha=0.2$	formatting-only ($v1 \rightarrow v2$)				criteria change ($v2 \rightarrow v3$)			
DeepSeek V4 Flash (primary)	—	9.4%	72.0% (4.50%)	87.8% (5.33%)	6.5%	12.3%	19.1% (4.17%)	52.3% (8.33%)	12.4%	18.9% (3.52%)	62.7% (7.50%)	
Qwen3.7-Flash	1.5%	17.2%	refused (135)	79.8% (14.35%)	3.1%	5.9%	48.4% (2.12%)	82.9% (4.05%)	6.6%	43.6% (3.22%)	85.7% (5.62%)	
Gemini 2.5 Flash-Lite	0.0%	8.6%	refused (50)	95.3% (8.22%)	0.2%	6.5%	23.3% (1.50%)	86.5% (5.87%)	5.1%	56.7% (2.73%)	88.7% (4.57%)	
Gemini 3.8 Flash	0.9%	4.6%	94.3% (4.35%)	95.5% (4.35%)	3.1%	4.2%	55.7% (1.93%)	84.3% (2.95%)	11.5%	24.1% (3.69%)	63.5% (8.11%)	
GPT-5 nano	5.8%	36.5%	refused (90)	refused (90)	16.3%	15.8%	refused (225)	30.0% (6.74%)	21.1%	refused (180)	21.0% (9.61%)	
GLM-4.7-Flash	9.4%	17.9%	67.3% (6.03%)	74.1% (6.29%)	4.4%	8.5%	36.5% (5.84%)	65.4% (6.15%)	9.3%	39.2% (1.26%)	64.9% (3.94%)	

and little reliability: 80.6% mean savings against 82.6%, certified in 98.7% of replications against 98.9%. At $\alpha=0.05$, where the default mode certifies in 74.4% of replications, the strict mode certifies in 4.5%. The two semantic edits behave the same way: within a point of the default mode at $\alpha=0.2$ (87.0% and 86.4% mean savings against 87.4% and 86.6%) and far below it at $\alpha=0.1$. On the less stable select column at $\alpha=0.2$ it certifies in 91.1% and 97.6% of replications but at 29.7% and 31.6% mean savings against 52.1% and 48.6%. At $\alpha=0.1$ it certifies in at most 2.7% of replications. Guaranteeing the stale subset alone is therefore cheap wherever a stratum's flip rate sits well below the budget, and its price rises steeply as the budget approaches the flip rate, one budget step before the default mode's own frontier. The strict mode's own certificates can exceed the budget too, at a rate the theorem allows. On SO scope widening at $\alpha=0.05$ it certifies in 0.4% of replications, each with a reuse-set rate above α , against $\delta_j = 0.05$.

n : The α -frontier and adaptive economics.

Savings collapse as α approaches a stratum's flip rate (Fig. 2), as the power analysis predicts; below that rate the certifier refuses. The doubling schedule prices both directions. A stratum whose sample rate exceeds the budget is refused at that look, the first one for the strata farthest from the budget. Near-frontier certifications escalate sampling only while the data stays consistent with certifiability. A single refusal near the

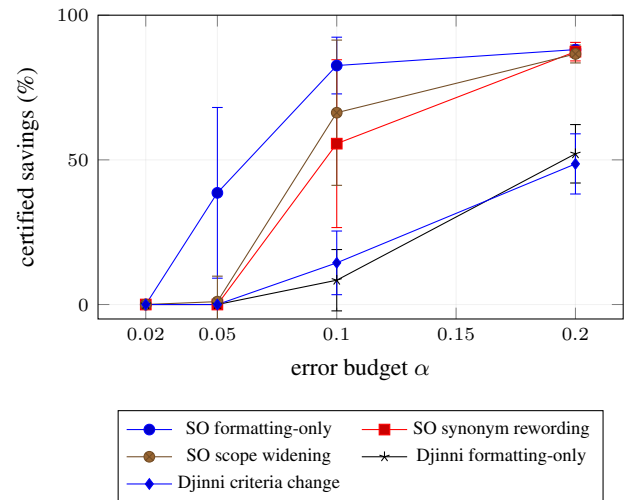


FIGURE 2. The α -savings frontier under the pinned procedure: mean $\pm$ one standard deviation over 1,000 sampling replications per point, generated from the result files. Savings collapse as α approaches each pair's flip rate.

frontier is not evidence that the stratum exceeds the budget. The synonym edit at $\alpha=0.1$ is refused on the main seed and certified in 81.4% of the sampling replications, a power miss the futility rule trades for its early stops.

o: Deployment scale.

To test the claim that sample cost is table-size-independent, we materialized the Boolean column across the *entire* 89,184-row corpus. That came to 78,956 billed cells in a single ledgered run costing \$9.31 at \$0.000118 per cell. The remaining 10,228 rows cost nothing, served by content-hash caching or already materialized by earlier passes. The run took 7.2 hours of wall clock at its 32-way concurrency, measured from its first cell write to its last. An estimate from stored latencies agrees: the column's 80,902 computed cells average 10.5 seconds per call, and those latencies, summed and divided by the concurrency, give 7.4 hours. We then certified the formatting edit against that materialized column with the real model as oracle, in three regimes over one sample path (Table 8). The budgets share that path at no cost in δ : the per-look bounds do not depend on α , so the event on which they all hold covers each budget. The first was live in August 2026 under the Maurer–Pontil bound of the preprint; that run issued the certificate the system applied. The second replayed the oracle draws stored by that run, under the exact bound. The third was live again on 29 September 2026, 55 days later, under the exact bound, against the same cache.

The August run. Theorem 1 does not cover this run, which used the Maurer–Pontil bound; it is reported because it is the certificate the system applied. Under Maurer–Pontil, at $\alpha=0.2$ the procedure spent 225 oracle calls to reuse 81,289 cells (91.1% fewer calls). It certified the cached-FALSE stratum at look 180 with 5 flips. Audited error was 3.16% over 2,686 audited reused rows. It refused the 7,715-row cached-TRUE stratum and so left for recomputation the 7,670 of its cells that the futility stop had not already drawn fresh. At $\alpha=0.1$ it escalated to 765 calls (25 flips in 720) for 90.5% savings. The run applied the loose certificate, persisting a `reuse_certificate` row and stamping its identifier on 79,130 of the 81,289 reused cells, which is what the table then served. The 2,159 it did not stamp already held a freshly computed cell when the certificate was applied, and the apply step refuses to write over one. Under the Bardenet–Maillard bound the loose certificate's evidence (5 flips in 180 draws from the 81,469-row cached-FALSE stratum) reads 0.230, which does not clear the budget at the look at which it was issued. The exact bound reads 0.073 on the same evidence, which clears $\alpha=0.2$ and $\alpha=0.1$ alike. The tight certificate's 25 flips in 720 read 0.055 under the exact bound, which clears $\alpha=0.1$, and 0.099 under Bardenet–Maillard, which clears it.

The same draws under the exact bound. Replayed over the stored August draws, the exact bound certifies the cached-FALSE stratum at look 45 (flips 1, bound 0.143). The whole certification therefore costs **90 oracle calls** and reuses **81,424 cells**. That is 91.3% fewer model calls (7,760 calls against 89,184), against a ceiling of 91.3% set by the cached-TRUE stratum the certifier never certifies. The procedure thus captures essentially all the reuse available to capture, and the residue is a property of the column. At $\alpha=0.1$ it certifies at look 180 (5 flips, bound 0.073) for 225 calls, the count the Maurer–Pontil run needed for the loose budget, reusing

81,289 cells (91.1% savings). At $\alpha=0.05$ the stored draws run out before the schedule's deepest look. That look reaches 1,440 rows of the stratum, 699 of which hold no August draw. Drawing them later would mix two snapshots, so the August evidence is silent there. Realized error was audited against oracle-computed cells of that snapshot only (status `done/cached`). Certificate copies are excluded by construction, as verbatim v1 values would register as non-flips. The audit found 89 flips in 2,821 audited reused rows, 3.15%. Of these, 1,812 rows belong to the released $n=2,000$ evaluation vector and show 3.20% on their own. The released subset is a consistency check with Table 2, and the remainder is independent evidence. A separate verifier script recomputes this audit, and the others in Table 8, from the row identifiers each run persisted and the stored cells. Priced at the ledger's measured \$0.000118 per cell, the maintenance workload would cost \$0.91 and about 42 minutes of wall clock on the latency estimate, against the \$9.31 the corpus cost to materialize. That estimate uses the materialization's mean per-call latency at the same 32-way concurrency. We price the recompute leg without executing it.

The September run: drift, measured. Assumption 1 scopes a certificate to the model snapshot its oracle draws came from. We re-ran the certification live on 29 September 2026: same model identifier, same gateway, same prompts, same cache, same seeded path, with the oracle draws stored under a version of their own. The snapshot had moved. We cannot tell a change of the provider's model from a change of its default settings (reasoning effort is left at the provider's default) or of how the gateway routes the same identifier; to the user they are one event. On the 720 cached-FALSE rows the August run had sampled, the edit's flip rate against the cache read 3.5% in August and 6.9% in September. Of those rows, 36 flipped only in September against 11 only in August (McNemar $p < 0.001$). A like-for-like comparison separates the model from the edit. The v1 prompt re-drawn in September disagreed with its own August draws, the cache, on 4.4% of those rows (Wilson [3.2, 6.2]). The same prompt's within-snapshot self-disagreement is 2.6% ([2.0, 3.1]). The v2 prompt re-drawn disagreed with its August draw on 6.5% ([4.9, 8.6]). The edit's flip rate with both sides drawn in September was 5.8% ([4.3, 7.8]). The ledger shows the same change: 534 output tokens per cell against 290 in August, at 24 seconds per call against 9. On the 45 cached-TRUE rows the edit's flip rate read 28.9% in August and 24.4% in September. Of those rows, 8 flipped only in August and 6 only in September (McNemar $p=0.79$). The v2 prompt re-drawn disagreed with its August draw on 31.1% of them ([19.5, 45.7]). The same stratum's floor on the lab column is 23.0%. These rows are too few to resolve a shift.

The certifier measures the sum of these effects against the cache it is asked to reuse, which is the quantity a user is exposed to, and it priced the drift as the theory says it should. At $\alpha=0.2$ it certified the cached-FALSE stratum at look 45 (2 flips, against 1 in August). In all it spent 90 calls and reused 81,424 cells (91.3%), in 101 seconds of wall clock including the oracle calls. The cached-TRUE stratum was refused at

TABLE 8. Deployment-scale certification of the formatting edit on the 89,184-row column, one sample path, three regimes. “FALSE look / flips” is the look at which the cached-FALSE stratum was decided and the flips observed there; audited error is the flip rate among reused rows that hold an oracle cell of that snapshot. The Maurer–Pontil rows are outside Theorem 1’s coverage (Section III) and are kept for the record: the certificate the system applied in August, and the same bound on the September draws.

Regime	oracle calls	FALSE look / flips	reused	savings	audited error (flips/rows)
<i>August 2026 snapshot, oracle draws of the live run (version 2)</i>					
Maurer–Pontil, $\alpha=0.2$ (live)	225	180 / 5	81,289	91.1%	3.16% (85/2,686) (applied)
Maurer–Pontil, $\alpha=0.1$ (live)	765	720 / 25	80,749	90.5%	3.03% (65/2,146)
exact, $\alpha=0.2$ (replay)	90	45 / 1	81,424	91.3%	3.15% (89/2,821)
exact, $\alpha=0.1$ (replay)	225	180 / 5	81,289	91.1%	3.16% (85/2,686)
exact, $\alpha=0.05$	not reachable from the stored draws (699 further draws needed)				
<i>September 2026 snapshot, oracle drawn live (version 4), same cache</i>					
exact, $\alpha=0.2$	90	45 / 2	81,424	91.3%	7.46% (104/1,395)
exact, $\alpha=0.1$	405	360 / 20	81,109	90.9%	7.96% (86/1,080)
exact, $\alpha=0.05$	135	90 / 6 (refused)	0	0.0%	–
Maurer–Pontil, $\alpha=0.2$	225	180 / 11	81,289	91.1%	7.54% (95/1,260)
Maurer–Pontil, $\alpha=0.1$	1,485	1,440 / 106 (refused)	0	0.0%	–

look 45 with 11 flips. At $\alpha=0.1$ it escalated to look 360 (20 flips, bound 0.091), one look deeper than the exact bound’s replay of the August draws. It spent 405 calls in all and reused 81,109 cells (90.9%). The 315 calls beyond the loose arm’s 90 took a further 304 seconds. At $\alpha=0.05$, where the August draws are silent, the futility rule refused the stratum at look 90 (6 flips, 6.7%) after 135 calls. Under Maurer–Pontil the same September draws certify the loose budget after 225 calls and are refused at the tight budget after 1,485 calls. Audited against every September oracle cell, the reused rows flip at 7.46% under the loose certificate and 7.96% under the tight one: inside both budgets, and well above the August figure of 3.15%, which is the drift once more. The audit covers 1,395 rows under the loose certificate and 1,080 rows under the tight one. The sample path needed 1,485 September oracle cells across the arms. Of these, 699 were already held from a pass earlier that day and 786 were requested from the runner. Of the requested cells, 20 were served by the content-hash cache and 766 were drawn fresh for \$0.15. That is \$0.000199 per cell against \$0.000118 in August. The longer outputs are the difference. The drift measurement drew 752 more. The refusal at $\alpha=0.05$ is the instrument reading correctly. The stratum’s flip rate against the aged cache exceeds that budget, so it was refused early, at look 90, and its cells are recomputed rather than served.

The comparison to the $n=2,000$ benchmark run shows that sampling cost did not scale with n . At $\alpha=0.2$ the call count was of the same order at both scales (90–180 calls, mean 97, against 2,000 rows; 90 against 89,184). This is because the doubling schedule stops on evidence about a stratum’s rate, not on its size, so its cost is bounded independently of the size. The call count is itself random, since a schedule stops at the first look whose bound clears α . At $\alpha=0.2$ the benchmark’s main seed spent 90 calls, with a replication median of 90, and at $\alpha=0.1$ the replications span 90–405 calls (mean 188, median 225). The scaling itself held: sampling cost is $O(1)$ in n while the certifiable population is $O(n)$, so savings rose from 88.4% to 91.3% on the comparable single-run measurement

(88.1% is the benchmark’s 1,000-replication mean, quoted elsewhere). Two reproducibility caveats apply. In MySQL the projection list changes which rows a seeded ORDER BY RAND returns, so the evaluation sample is defined by an exact query string, not by the seed alone. And the certifier at the time of the August run shuffled each stratum in the order the cell query returned, which the query planner served through one index in August and another afterwards. The August run walked the content-hash order, the certifier now sorts by content hash before shuffling, and the replay reproduces the August run’s every count under the Maurer–Pontil arm (the released script asserts it).

p: The stratification hierarchy.

Aggregate-only certification (one stratum) is valid for its aggregate estimand yet conceals 35.3% realized error in the reused cached-TRUE subgroup on the formatting edit at $\alpha=0.1$ and 44.6% on the widening edit at $\alpha=0.2$. It buys its aggregate with the stratum whose floor on the lab column is 23.0%, which per-value strata quarantine by construction. Refining further by embedding-interaction tertiles helps only when within-value risk is heterogeneous and otherwise costs power through the δ/K split. On the headline configuration the value $\times$ embedding-tertile refinement certifies strictly less than value strata alone (82.7% against 88.1% savings on SO formatting at $\alpha=0.2$), because the split outweighs its discrimination gain. Across all 40 shared configurations of the released ablation the refinement is better in 6 and worse in 22, with 12 ties where neither certifies. Every win sits at a budget where value strata alone certify in at most 2.7% of replications. The largest, SO scope widening at $\alpha=0.05$, lifts mean savings from 1.0% to 5.9% by certifying in 33.3% of replications against 1.2%. It does so because the finer strata isolate a slice of a value stratum whose rate clears a budget the whole stratum does not. We report this as a negative result instead of tuning until it inverts. The refinement’s validity is unconditional (Assumption 2), its power is not, and on these columns within-value risk is homogeneous enough that the

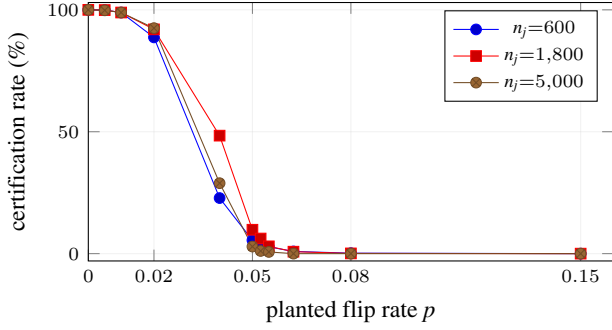


FIGURE 3. Calibration power at $\alpha=0.05$ (presented-cells estimand, 2,000 trials per point): certification rises smoothly as p falls below α and is small but nonzero just above it, where the realized population rate of a planted stratum often falls below α and certifying it is correct; the unconditional rate of certificates whose realized error exceeds α stays at or below 1.00% in every configuration against the nominal 5%. The $n_j=1,800$ curve sits above the other two at intermediate p because the exact bound's finite-population correction tightens as the sampled fraction grows: the six-look schedule reaches most of an $n_j=1,800$ stratum but a minority of an $n_j=5,000$ one, and the $n_j=600$ stratum's final look is the whole stratum, which cannot certify, so its deepest certifying look is its fourth.

δ/K split is repaid only where the value strata certify almost nothing.

q: Minority-stratum anatomy.

The cached-TRUE stratum of the Boolean column flips 37.1% under a formatting edit. A majority vote of three draws on both sides (vote-of-3) reduces this to 24.7% and shows that 20.4% of cached-TRUE values were single-draw noise. The flips that survive vote-of-3 are genuine boundary volatility. Quarantine-and-recompute is therefore the correct systemic response, and the actionable upstream knob is vote-based computation of boundary-stratum cells at write time.

r: Calibration.

The calibration study (Fig. 3) covers planted flip rates across $p \in [0, 0.25]$, budgets $\alpha \in \{0.05, 0.1, 0.2\}$ (including the one every headline result uses), stratum sizes $\{600, 1,800, 5,000\}$, both estimands, and 198 configurations (396,000 runs). The quantity Theorem 1 bounds is the rate of unsafe certificates: certifications of a stratum whose realized flip count exceeds αn_j (in strict mode, $\alpha(n_j - m_j)$). The worst such rate over all configurations is 1.00% (95% CI [0.65, 1.54]%) against the per-stratum $\delta_j=5\%$ the study plants. It occurs at $\alpha=0.05$, $p=0.0525$, $n_j=1,800$, in the default mode. The presented-cells rate, which leaves out the sampled flips, exceeds α in at most 0.70% of runs. Of the configurations, 54 are *tight* nulls, planted just above the budget at 1.05 to 1.25 times α , where the futility rule often cannot dispose of the stratum at the first look and the sampling fraction climbs. Over these the unsafe rate averages 0.35%. Their certification rate reaches 6.2%, and most of those certificates are correct. At p just above α the realized population rate of a planted stratum falls below α in a large share of trials, and the exact bound certifies what is actually there. Planting by rate is therefore

not the least favorable case. The theorem's event is hardest to avoid for a fixed population whose realized count sits one flip past the budget, and there the probability of an unsafe certificate is an exact finite sum over the schedule. Across the 9 (budget, size) combinations of the grid it lies between 1.3% and 2.6% in the default mode and reaches 1.0% in the strict mode, inside $\delta_j=5\%$ in each. The remaining gap comes from the Bonferroni split across looks (the earliest of which cannot certify at a tight budget), from the discreteness of the test, and from the futility rule, which stops sample paths that a later look might have certified. The reuse-set rate evaluated on default-mode certifications fluctuates past α in frontier configurations. The worst is 100.0% of certified runs at $p=0.0625$, $\alpha=0.05$, certified 0.80% of the time. That is expected, since the default mode does not certify that metric. The strict reuse-set mode holds its own guarantee within δ_j (unsafe certificates in at most 0.60% of runs), with certification power rising smoothly as p falls below α and with stratum size.

s: Independence across rows.

Assumption 1 asks that oracle draws be independent across rows, and provider-side batching could in principle correlate concurrent requests [8], [9]. We tested the consequence the proof needs, non-interference, on 300 rows of the evaluation vector, drawing the formatting edit's new version twice for every row. The draws were made on 29 September 2026, once with a single request in flight (78 minutes of wall clock) and once at the deployment run's concurrency of 32 (4.6 minutes), against the same cache. Requests that errored or exceeded a 90-second ceiling are excluded from their arm: 9 sequential and 28 concurrent, where the ceiling binds more often. On the 263 rows both arms kept, the rates are 9.5% sequential and 8.4% concurrent. Of those rows, 14 flipped only sequentially and 11 only concurrently (exact McNemar $p=0.69$). The excluded rows show no sign of being flip-prone, on small counts. Of the 28 rows the concurrent arm dropped, 1 flipped in the sequential arm. Of the 9 the sequential arm dropped, 2 flipped in the concurrent arm.

t: The planned tests.

The script fixed three analyses before the run, each reported against its reference distribution at the 5% level. The first is a two-proportion test on the flip rates. The second sets the two arms' disagreement on the same row against the August stratum floors. The third looks for serial dependence in completion order, with a lag-one permutation test and a dispersion test of flip counts over consecutive blocks of 32 completions. The rates agree: 8.9% (Wilson [6.2, 12.8]) sequential against 8.8% ([6.0, 12.8]) concurrent, two-proportion $p=0.96$, a test whose minimum detectable difference at 80% power is 7.0 percentage points. The two arms disagree with each other on 9.5% of rows ([6.5, 13.7]): 9.2% of 238 cached-FALSE rows and 12.0% of 25 cached-TRUE rows, against August floors of 2.6% and 23.0%. After the drift measured above, that second comparison cannot separate concurrency from the change of

snapshot, and we draw nothing from it. In completion order, the lag-one autocorrelation of the flip indicator is -0.006 in the concurrent arm (permutation $p=0.95$) and 0.070 in the sequential arm ($p=0.27$). The variance of flip counts over consecutive blocks of 32 completions is 2.08 times its binomial value in the concurrent arm (8 blocks; 95% CI [0.91, 8.62], which covers 1; upper-tail chi-square $p=0.04$). In the sequential arm the ratio is 1.39 (9 blocks; $p=0.20$).

u: Reading the tests.

The rate and lag tests do not reject. The dispersion test rejects one-sided at the 5% level in the concurrent arm ($p=0.04$), on 8 blocks. To add blocks we drew a concurrent arm alone on a further 1,000 rows of the vector on 30 September 2026 (31 blocks; 0 requests failed or hit the ceiling). Its block dispersion is 1.26 (95% CI [0.81, 2.26]; upper-tail $p=0.15$) and its lag-one autocorrelation -0.031 ($p=0.34$). Its flip rate is 7.1% ([5.7, 8.9]) against the sequential arm's 8.9% ($p=0.30$). This arm was drawn a day later, on other rows, and under faster serving (a median response of 3.7 seconds against 14.4 in the first concurrent arm), so the comparisons between arms also span a day of the snapshot. Pooled over both concurrent arms, the dispersion statistic is 52.5 on 37 degrees of freedom ($p=0.048$). The block-scale question is therefore bounded, not settled. The larger arm alone does not reject. The pooled test, which includes the first arm, rejects narrowly, and ratios up to 2.26 lie inside the larger arm's interval.

Two looks that were not in the plan argue for caution. Flips concentrate in slow responses: in the larger arm, 59 of the 500 slower responses flip against 12 of the 500 faster ones. Completion order is therefore not neutral with respect to the outcome. With blocks formed in launch order instead, the dispersion ratio is 1.61 in the larger arm ($p=0.019$) and 1.82 in the first ($p=0.08$). The ordering was chosen after seeing the data, so we report these as cautions, not as findings.

What the check supports is narrower than independence. The flip rate, the quantity the bound estimates, does not move with concurrency in these draws; a difference smaller than 7.0 percentage points is not excluded. Dependence among draws issued together is not excluded either, and it would raise a certificate's error probability. We simulated the certifier on the deployment stratum, at its two certified budgets and at a flip rate one flip past the budget, with beta-binomial dependence inside blocks of 32 draws. With independent draws its per-stratum error probability is at most 2.3%. It rises to 4.1% at the planned test's dispersion estimate in the larger arm, 7.1% at the launch-order estimate, and 12.2% at the upper end of the planned test's interval, against a nominal $\delta_j = 0.05$. That exposure is confined to strata whose flip rate sits just above the budget. At the loose budget the deployment stratum is not one: its audited error is 3.15% in August and 7.46% in September (Table 8). At the tight budget the September audit, 7.96% against $\alpha=0.1$, leaves less room. Two further things limit the exposure. First, the replication results of this article resample frozen labels, a fixed population for which the hypergeometric law holds however the labels were

drawn; the assumption matters for a live certification, as in the deployment runs. Second, a certification sample is small (a few hundred calls in those runs), so a deployment that cannot accept the assumption can issue the sample's calls one at a time, which costs wall clock and no additional calls. The check covers one model through one gateway at one concurrency, in the September snapshot.

v: The bound ablation.

Every result above except the Maurer–Pontil rows of Table 8 spends its δ through the exact bound of Eq. (1), split across looks by Bonferroni. The bound, not the stratification or the schedule, is what sets the frontier. We measured how much by swapping it inside the same pinned procedure over the same stored labels (Table 9; Appendix B lists every configuration). On a clean sample at $n=90$ the exact bound reads 0.050 and its binomial limit, the Clopper–Pearson bound [11], 0.052. The Maurer–Pontil empirical-Bernstein bound [12] that the preprint certified with reads 0.144, and the Bardenet–Maillard without-replacement bound [13], [22] reads 0.317. A Waudby-Smith–Ramdas betting confidence sequence [14], which is anytime-valid and therefore spends the stratum's δ_j without splitting it across the six looks, reads 0.065.

w: The bounds compared.

End to end across the 20 configurations of Table 2 (main seed), the exact bound certifies 10 of them against 7 under Maurer–Pontil and 7 under Bardenet–Maillard. Mean savings are 29.5% against 18.5% and 16.4%, on 7,458 oracle calls against 11,515 and 12,325. Relative to the exact bound, Maurer–Pontil loses 3 certificates and gains 0, and of the shared ones it reaches 1 at the same look and 6 at a later one. The widest single gain is SO scope widening at $\alpha=0.1$, from 18.1% savings for 1,485 calls to 72.0% for 405. The empirical-Bernstein bounds pay a range term that a binary loss does not need, and the without-replacement form pays more, not less, because its constants are larger. The preprint reported it as tighter. Its replay used the Maurer–Pontil linear constant inside the Serfling form, and the published constant, $\kappa = 7/3 + 3/\sqrt{2}$ [13], is what the numbers here use. The Clopper–Pearson limit certifies 9 configurations at mean savings 28.1% on 8,054 calls, the finite-population correction buying the exact bound its one further certificate. The betting sequence certifies 9 configurations at mean savings 29.1% on 7,616 calls. It beats the exact bound on 1 of the 20 configurations, where the schedule's Bonferroni split bites hardest (Djinni criteria change at $\alpha=0.2$: 76.3% against 62.7%). It loses to it on 2 and matches it on the rest (certificates lost 1, gained 0). Its validity here rests on a capital process built for draws whose conditional mean is constant, which without-replacement draws do not have. The without-replacement variant given in [14], which we did not implement, or a time-uniform boundary [15] would remove the split without that caveat and is the remaining power lever we see.

Validity comes before power, so all five arms were replayed through the calibration protocol (36,000 trials per arm, 18,000

of them at planted rates above the budget). At planted nulls the exact bound certifies up to 6.3% of trials, against a per-stratum δ_j of 5%. Most of those certificates are correct, because the realized count of a planted stratum falls below αn_j in many trials. The certificates whose realized count exceeds the budget, the event of Theorem 1, are at most 1.00% of trials in any null configuration ($\alpha=0.1$, $p=0.105$, $n_j=1,800$; Wilson [0.54, 1.83]%). Of the 18 null configurations, 0 carry a certification-rate interval above δ_j . The Clopper–Pearson limit certifies 1.5% of null trials at worst (0.50% unsafe), and the betting sequence 2.5% (1.20% unsafe). Maurer–Pontil and Bardenet–Maillard certify 0.0% and 0.0% (0.00% and 0.00% unsafe), leaving almost all of their δ_j unspent. A clean 1,800-cell stratum at $\alpha=0.05$ costs the exact bound 90 draws (180 at $n_j=5,000$ in the calibration study). It costs the Clopper–Pearson limit 180, the sequence 180, Maurer–Pontil 360, and Bardenet–Maillard 720. The frontier in Fig. 2 is therefore a property of the bound. The exact bound is the pinned one because it is proven for exactly the sampling the procedure performs and, on every configuration measured here, is the tightest of the proven arms. The preprint’s certification results are reproducible by running the released harness with the Maurer–Pontil arm selected.

x: Embedding-transfer operating points.

A measured τ sweep over per-row similarities shows the guarantee-free transfer baseline collapses within a threshold window about 0.02 wide. On the formatting edit it collapses from 100% of rows reused at $\tau=0.96$ to 10.7% at 0.98 and none at 0.99. On the widening edit it collapses from 99.9% at 0.94 to 8.1% at 0.96 and none at 0.97. Decisively, realized error among reused rows stays at the population flip rate at every operating point (5.6%–6.6% and 8.0%–9.8%): the threshold controls volume, never error, because cross-version similarity carries no per-cell flip signal (B1).

VI. THE PROMPT-EDIT BENCHMARK

Semantic benchmarks exist, but they benchmark *engines* on a fixed task [23]; none exercises maintenance across prompt versions, which needs a different artifact: version pairs plus per-cell flip labels against the new version. Ours comprises five components. The first is two redistributable corpora: the Stack Overflow Developer Survey (89,184 rows, ODbL 1.0 with the Database Contents License, DbCL 1.0) and Djinni English candidate profiles (210,250 rows, MIT). The second is three materialized columns of two types: two Boolean columns over structured facets, one of them the production column and one a lab replica seeded from its second version; and one four-way ordinal select over long free text. The third is five realistic edit pairs spanning formatting-only edits, synonym rewording, scope widening, and a criteria change. The fourth is full ground-truth flip labels for $n=2,000$ rows per version pair at pinned decode, the three-draw relabeling of the headline pair (which also supplies its self-flip floor), and the select column’s self-flip probe. The labels and the relabeling draws are 28,000 labeled cells in all: 16,000 distinct version

cells and 12,000 relabeling draws. The fifth is the deterministic harness (seeded sampling, adaptive certifier, baseline policies) that reproduces every number in this article. Table 10 lists the five pairs with their exact deltas. Ledgered model spend across the entire evaluation is \$12.01 (plus 146,753 cells from direct-call probes the ledger does not see), so the benchmark is cheap to extend to budget-class models.

VII. RELATED WORK

Table 11 positions sIVM against the lines of work it borrows from, is compared with, or is mistaken for, by the quantity each certifies and the axis of change it holds fixed.

a: Guaranteed semantic query processing.

Semantic-operator systems bound the error of *one-shot* execution against a reference model. They include LOTUS’s optimized operators with accuracy targets [26], SUPG’s proxy-based selection with precision/recall guarantees [10], and its aggregation counterpart ABae, which stratifies by proxy before spending an expensive predicate [24]. They also include BARGAIN’s adaptive cascade thresholds [25], ThalamusDB’s approximate natural-language predicates with error bounds [27], and pipeline-level accuracy budgeting in Task Cascades [28]. All certify a *fixed* definition per query; none transfers evidence across a definition change. We reuse their statistical framing (a labeled sample, a bound, a threshold) in a maintenance loop those systems do not model, with a different bound. The loss is binary and the sample is drawn without replacement from a frozen stratum. The exact finite-population bound, the hypergeometric form of Clopper and Pearson’s binomial limits [11], is therefore available and dominates the empirical-Bernstein bounds those systems and the preprint of this article relied on (Section V). Time-uniform confidence sequences [14], [15] are the alternative that would remove the split across looks. The certificate itself is distribution-free risk control in the sense of risk-controlling prediction sets [39] and Learn then Test [16]. It is an upper confidence bound on a risk that must clear a user-set α with probability $1 - \delta$, here evaluated per stratum and per look with the multiple-testing accounting those frameworks make explicit. The declarative LLM data systems that optimize cost and quality without a statistical target are Palimpsest’s AI-powered analytics [40], DocETL’s agentic pipeline rewriting [41], Evaporate’s structured views over data lakes [42], and UQE’s sampling-driven queries over unstructured collections [43]. They compute the same kind of prompt-defined per-record outputs. None certifies which of its outputs survive a redefinition.

b: Verified caching.

vCache certifies that a *new query* may reuse a cached answer under one fixed definition, with learned per-entry thresholds and user error rates [29]. Evergreen spends confidence sequences and early stopping to cut the cost of *verifying* semantic aggregates, not to decide reuse [31]. FreshCache approves a cache hit only when a learned staleness model

TABLE 9. Five upper confidence bounds inside the same pinned procedure on the same stored labels: the bound on a clean sample of 90 draws and on the deployment evidence of 5 flips in 180 draws (the four fixed-sample bounds at the pinned per-look level, $\delta/12$; the anytime-valid betting sequence at the unsplit per-stratum level, $\delta/2$); the certifying configurations, mean savings, and oracle calls over the Table 2 grid (main seed); and the worst rate of unsafe certificates, the event of Theorem 1, at planted nulls in the calibration replay (per-stratum $\delta=0.05$). WoR: without replacement.

Bound	upper bound on		Table 2 grid (20 configurations)			unsafe at nulls	
	0/90, $N=1,800$	5/180, $N=81,469$	certifying	mean savings	oracle calls		worst
Exact finite-population (pinned)	0.050	0.073	10	29.5%	7,458		1.00%
Clopper–Pearson (binomial)	0.052	0.073	9	28.1%	8,054		0.50%
Betting confidence sequence	0.065	–	9	29.1%	7,616		1.20%
Maurer–Pontil (preprint)	0.144	0.140	7	18.5%	11,515		0.00%
Bardenet–Maillard (WoR)	0.317	0.230	7	16.4%	12,325		0.00%

The betting sequence has no 5/180 entry because its bound is a function of the whole draw sequence, not of (k, n) alone.

TABLE 10. The five versioned prompt-edit pairs. Deltas are transcribed from the stored prompt templates; bound fields are the row attributes the template interpolates. Flip rates are single-fresh-draw labels against the cached version on the $n=2,000$ evaluation vector (Table 2); the SO synonym pair is measured against the formatting version’s cells ($v2 \rightarrow v3$), as the benchmark defines it.

Pair	Column (corpus; contract; bound fields)	Exact delta	flip
SO formatting-only ($v1 \rightarrow v2$)	Data-platform specialist, lab replica (SO; Boolean; role, databases, platforms, tools, other tech)	Two paragraph breaks inserted after the question and before the “Judge from” sentence; “OR heavy analytics engineering” lowercased to “or heavy analytics engineering”; no content word changed	6.3%
SO synonym rewording ($v2 \rightarrow v3$)	same	“this person” $\rightarrow$ “this individual”; “works primarily on” $\rightarrow$ “chiefly builds and maintains”; “heavy analytics engineering” $\rightarrow$ “intensive analytics engineering”; “Judge from role ... and other tech” $\rightarrow$ “Decide using their role ... and other technologies”; “Working with multiple ... counts as evidence” $\rightarrow$ “Experience with several ... is supporting evidence”	8.9%
SO scope widening ($v1 \rightarrow v2$)	Data-platform specialist, production column (SO; Boolean; same fields)	Definition extended from “warehouses, lakes, pipelines, distributed data stores, or ML data tooling” by “OR heavy analytics engineering”; the exclusion “Merely using one relational database for app work does not qualify” replaced by the inclusion “Working with multiple analytical/distributed databases (BigQuery, Snowflake, Spark, Kafka, Elasticsearch) counts as evidence”	9.4%
Djinni formatting-only ($v1 \rightarrow v2$)	Seniority tier, lab (Djinni; four-way ordinal select; position, years of experience, CV text)	The one-line template broken into a question line, three labeled input lines (Position, Years of experience, CV), and the instruction sentence, with no word changed	12.3%
Djinni criteria change ($v2 \rightarrow v3$)	same	Sentence appended: “Evidence of team or people leadership (leading engineers, owning hiring, running a team) moves the candidate up one tier”	12.4%

puts the probability that the entry is stale below a per-tier error budget [30]. FreshCache is the closest in shape, since a per-partition budget gating reuse is exactly our skeleton, and it differs on the two axes that decide the guarantee. Its risk is change-of-world (open-web evidence aging under fixed query semantics) where ours is change-of-definition. Its per-tier quantity is a fitted model’s point estimate where ours is a finite-sample upper confidence bound that either clears α or refuses. Our B1 baseline shows why these mechanisms do not port: across a definition change the query-to-entry similarity that such caches threshold carries no per-cell flip signal (AUC 0.454 and 0.499). sIVM certifies change-of-definition transfer, the axis those systems hold fixed; the two compose (a certificate is scoped to a snapshot, and change-of-world machinery decides when the snapshot itself expires).

c: Incremental and adaptive view maintenance.

Sampling a stale materialized view to bound the error of answers served from it [32] is the closest classical relative of what we do, and the difference is what goes stale. There the data drifts under a fixed view definition; here the definition itself changes over fixed data. That is why that work’s estimators condition on updates and ours on a prompt delta. The look schedule itself is online aggregation’s template [33], a running estimate whose interval tightens as sampling proceeds, repurposed to certify a stratum rather than to answer an aggregate. Classical incremental view maintenance (IVM) propagates base-data changes through a fixed view definition [34], and view adaptation after redefinition [35] rewrites the materialization when the definition changes. Both assume deterministic operators, where a rewrite either preserves a

TABLE 11. What neighboring systems certify, and across what. sIVM is the only row whose certified quantity is the reuse of cached outputs across a change of the prompt definition.

System or line	Certified quantity	Held fixed	Changes	Guarantee form
SUPG, ABae, BARGAIN, LOTUS, ThalamusDB, Task Cascades [10], [24]–[28]	precision, recall, or accuracy of one query’s answer against an oracle	the query definition	nothing (one-shot execution)	finite-sample bound per query
vCache [29]	correctness of serving a cached answer to a <i>new</i> query	the answer semantics	the incoming query	learned per-entry threshold at a user error rate
FreshCache, Evergreen [30], [31]	probability that a cached entry is stale (world drift); cost of verifying aggregates	the query definition	the world the answer describes	fitted staleness model per tier; confidence sequences
Stale-view cleaning, online aggregation [32], [33]	error of an answer served from a stale view; a running aggregate	the view definition	the base data	sampling bound on the answer
Classical incremental view maintenance (IVM) [34]	exact equality of the maintained view with recomputation	the view definition	the base data	algebraic (no sampling)
View adaptation after redefinition [35]	exact equivalence of the rewritten view	deterministic operators	the view definition	algebraic (no sampling)
Step-level reuse [36]	per-step equivalence under a perturbed request	the model	the request	deterministic per-step check
Guarantee-free semantic caches [21] (B3)	nothing: a hit is a similarity above a threshold	nothing	the incoming query	none
Per-output verification: assertions, regression suites, majority vote [37], [38]	correctness of one output against a checker or a majority	the prompt	the output under test	per-output check, no population bound
sIVM (this work)	false-reuse rate of cached stochastic cells kept under the new definition, per stratum	the model snapshot and the data	the prompt definition	exact finite-sample bound per stratum and look

subexpression or does not. Stochastic LLM cells break the premise. Our measurements show that even a semantically null rewrite flips cells at about the column’s noise floor (6.3% against 5.0%), so “preserved” is not a property a rewrite can have here, only a rate one can bound. Streaming semantic systems recompile operators mid-stream but recompute outputs instead of certifying reuse of old ones [44], [45]. Streaming cascades certify on the *data-arrival* axis, where new tuples meet a fixed operator, and leave the definition axis open [46]. Given that distinction, sIVM is, to our knowledge, the first maintenance procedure whose unit of transfer is a *statistically certified set of stochastic cells*.

The nearest counterexample in shape is step-level reuse with lightweight verification and selective patching across perturbed requests [36], which does reuse computation when a request changes. It differs on exactly the property this article is about: its verification is a deterministic per-step check, so it carries no error budget and makes no population statement about the set it reuses. In-database prompt management [47] and versioned, provenance-tracked prompt views [48] supply the substrate this work assumes, not a competing mechanism. We take the framing of prompts as versioned views from that

line and claim only the certification machinery built on top of it, whose reuse is of *outputs*, not of key-value attention state.

d: Prompt sensitivity and regression.

Prompt-delta analysis for assertion synthesis [37] anticipates our edit taxonomy but targets pipeline quality checks, not reuse certification; interactive LLM regression tools frame migration pain without guarantees [38]. Our finding that surface edit class can fail to predict flip rate (a synonym paraphrase flipping more cells than a formatting edit; a formatting edit indistinguishable from a criteria change) shows why measurement, not taxonomy, must carry the decision.

VIII. LIMITATIONS AND OUTLOOK

a: Which budgets are usable, and for what.

Most of the savings in Table 2 sit at $\alpha=0.2$. This budget allows up to one cell in five of a certified stratum to flip on recomputation (with probability $1 - \delta$ over the run, in the realized terms of Theorem 1). That is a defensible budget where a person reads the column row by row before acting on it (a recruiter screen, a lead list, an enrichment sheet a reviewer walks through). Two further conditions apply: the

cost of a stale cell is a wasted glance, and the alternative on offer is today's status quo, which reuses everything at the population flip rate with no bound at all. The same budget is not defensible for a column that feeds an aggregate or a downstream filter. There a tighter budget and the strict reuse-set mode are the right setting, and Section V shows what they cost. On the stable Boolean column the default mode certifies 82.6% mean savings at $\alpha=0.1$ and certifies in 74.4% of replications at $\alpha=0.05$. The strict mode certifies 80.6% mean savings at $\alpha=0.1$ and certifies in only 4.5% of replications at $\alpha=0.05$. On the less stable select column the default mode still certifies at $\alpha=0.1$ in most replications, and the strict mode in at most 2.7%. There $\alpha=0.05$ is refused in both modes, because the column's flip rates sit above it. At deployment scale the sample a tight budget needs is still a constant, 405 calls at $\alpha=0.1$ against 89,184 cells, so tight budgets are affordable in calls. What they cannot buy is certification of a stratum whose flip rate exceeds them, which is the correct outcome. A practitioner should therefore read the frontier of Fig. 2 against the column's floor and choose the budget the downstream use tolerates, not the one that maximizes savings.

b: Statistical scope.

Bonferroni across strata and looks is sound but conservative, and Section V prices it. The exact bound leaves a gap between its measured unsafe-certificate rate and the nominal δ . A betting confidence sequence, which spends no split across looks, beats it where that split bites hardest and matches or loses to it elsewhere. The without-replacement betting sequence of [14] or a time-uniform boundary [15] would keep the exact bound's validity under without-replacement sampling while removing the split across looks. E-value selection (the e-value Benjamini-Hochberg procedure) [49] composes with either. None of these is needed for the guarantees stated here. The independence of oracle draws across rows is an assumption that our check bounds but does not establish (Section V); a live certification under concurrency inherits whatever dependence the serving path introduces. Our stratifiers are first instances; the lexical variant is weak on broad edits, and the embedding variant's signal exists only for semantic edits, so learned interaction models are an open lever with guaranteed validity (Assumption 2).

c: Equivalence and columns.

In the main evaluation equivalence is exact match, and the columns are two Boolean columns and one four-way ordinal select; numeric and free-text columns, which the introduction's product examples also serve, are not evaluated there. The five author-written edit pairs, one per class per column, are enough to show that surface class can fail to predict flip rate, not to estimate how often it does. The edit-class findings should be read at that strength. Because both Boolean columns ask one question over one corpus, the observation that a column's own stability dominates the edit rests on two columns. A free-text column needs a judge, and the judge's

own error enters the budget. In a stratum, let q be the share of pairs the judge calls equivalent although a reader would not (missed flips). The flip rate the judge reports for the stratum is then at least the true rate minus q , and q is at most the judge's miss rate among the pairs it calls equivalent. So let $\hat{m}$ be an upper confidence bound on that conditional miss rate at level δ_{cal} , from human-labeled pairs of the same column, and certify at $\alpha - \hat{m}$. A certified stratum's true rate is at most α whenever the calibration bound and the certificate both hold, provided the judge's conditional miss rate in the certified stratum is no higher than in the calibration pool. They both hold with probability at least $1 - \delta - \delta_{\text{cal}}$. Errors in the other direction (calling equivalent cells different) only raise the reported rate and cost power. The calibration belongs to the (edit, column) pair it was drawn from, or to a pool of comparisons on one column if their conditional miss rates are taken as equal, since a judge's miss rate is a property of the outputs it compares. The machinery of this article is otherwise unchanged, but its power under a noisy judge has one measurement so far.

d: A free-text pilot.

We ran the recipe once, on a free-text column over 2,000 SO profiles that names each respondent's primary technical specialty in a short phrase. The column has three prompt versions: a baseline, a formatting-only edit of it, and a synonym rewording. The judge is the primary model (deepseek/deepseek-v4-flash-0731), asked in both orders whether two phrases name the same specialty. A pair counts as equivalent only when both orders say so. A pair either of whose requests failed (errored or timed out) is excluded. The failures came in bursts of consecutive rows and did not recur on the same rows across versions. The exclusion leaves 1,602, 1,821, and 1,799 of the 2,000 pairs usable for the floor, the formatting edit, and the synonym edit. Under that judge the column's own floor, two draws of the same prompt, is 40.1% (Wilson [37.8, 42.6]). The formatting edit flips 42.2% ([40.0, 44.5]) and the synonym edit 49.9% ([47.6, 52.2]).

e: Pilot calibration.

We hand-labeled 201 calibration pairs (one labeler, the author), drawn evenly from the three comparisons and pooled: 150 the judge had called equivalent and 51 it had called different. Each pair was shown in a shuffled order as two phrases with a Same/Different choice, without the judge's verdict or the comparison it came from. The judge missed 0 true differences among the 150, an upper confidence bound of 2.0% on its miss rate among the pairs it calls equivalent at $\delta_{\text{cal}}=5\%$. The bound is pooled across the three comparisons; each comparison's 50 labels alone give 5.8%. The judge also called 41 equivalent pairs different. A second model (meta/muse-spark-1.3-contributor) labeled the same pairs and agreed with the hand labels on 74.1% of them; the judge itself agreed on 79.6%. The second model called 20 of the 150 judge-equivalent pairs different, every one of them

a pair the author called the same. Under its labels the miss bound would be 18.8%, so the clean side of this calibration rests on the author's reading.

f: Pilot outcome and reading.

Certifying at $\alpha - \hat{m}$ takes 2.0 percentage points off each budget under the author's labels: $\alpha=0.2$ becomes 0.180 and $\alpha=0.1$ becomes 0.080. Under the second labeler's labels the deflation would be 18.8 points, more than the tight budget itself. Stratification is by the cached phrase's category (8 categories, assigned once from the cache by the same model). In that configuration, both edits are refused on every stratum at both budgets, each run stopping after 360 oracle calls (every stratum at its first look). The composed guarantee holds with probability $1 - \delta - \delta_{\text{cal}}$ under the assumption stated above; the judge's price is the deflation plus the labels. The floor the certifier met is the judge's, not the model's. Under the author's labels, 41 of the 51 pairs the judge called different name the same specialty, across the three comparisons, and 0 of the 150 pairs it called equivalent differ. Weighting the judge's verdicts by those label shares puts the disagreement of two same-prompt draws, as the author reads them, near 7.9% (band [4.4, 14.5]% from the Wilson intervals of the two shares), inside the loose budget. The two labelers read the judge differently. By the author's labels it is stricter than the author's reading and misses nothing. By the second labeler's labels it is mostly right when it calls a pair different (42 of 51) and misses 20 of the 150 pairs it calls equivalent. One human labeler and one model cannot say which reading is right; a second human labeler would separate a strict instruction from a lenient labeler. Either way the certifier, which sees only the judge, refused every stratum. The composed guarantee holds for any judge whose miss rate the calibration bounds correctly. What the pilot does not deliver is a judge whose equivalence matches a reader's, calibrated in both directions.

g: Models and snapshots.

One budget-class reasoning model carries the full experimental program. A further 6 families were run on the lab column's two pairs and 5 of them on all five (Section V). In those runs at least one stratum certified on 6 of the 6, and the same edit's flip rate ranged widely. The quantitative frontier (where savings sit for a given α , and which edits are benign at all) is therefore specific to the (edit, model) pair. The qualitative structure nevertheless held on every family: a measurable floor, which limits what certifies when an edit leaves the output distribution unchanged, and a certifier whose refusals track the measured flip rate. Certificates are scoped to the model snapshot their oracle draws came from. Section V shows what that scope is worth. The same model identifier through the same gateway had drifted 55 days after the deployment run. The certifier measured the drift as a higher flip rate against the old cache. It re-certified the two budgets the August run had certified, one of them a look deeper than the exact bound's replay of the August draws. It refused the tightest budget, on which the August evidence

is silent. Detecting, without a full re-certification, when a snapshot has moved, i.e., composing sIVM with change-of-world invalidation [30], [31], is the natural next step.

h: Economics.

The economic frame that matters in production is time-to-consistency under provider rate limits. In that frame certified maintenance samples 5%–74% of cells across the configurations of Table 2 that certify in most replications (5% on the headline formatting edit at $\alpha=0.2$) and recomputes only the strata it refuses, instead of recomputing 100%. That is a wall-clock and rate-limit gain that grows with table size, because sample sizes do not. The live re-certification of Section V measured the certification decision at 101 seconds of wall clock for the loose budget. This article does not measure two things. The certificate surface in the UI was not studied with users, so whether a person acts differently on a column that shows one is open. And the cost figures come from one ledgered materialization and one priced maintenance replay at the gateway's list rates; they show the scale of the saving, not a cost model.

IX. CONCLUSION

Prompt-defined columns are materialized views whose *definition* changes far more often than their data, and the systems serving them today choose between recomputing everything and reusing everything. sIVM shows the middle is reachable and cheap: freeze a stratification, sample on a doubling schedule, bound each stratum's flip rate with an exact finite-population bound, reuse only what clears the budget. At deployment scale that cost 90 oracle calls to certify reuse of 81,424 of 89,184 cells. The sample did not have to grow with the table, because the schedule stops on evidence about a *rate*, at a cost bounded independently of the size. These results come from one primary model on Boolean and select columns. On 6 of 6 further model families at least one edit certified, and which edits did so varied by family.

The result we did not anticipate is that the certifier doubles as a measuring instrument for the column it maintains. A stratum refused at every budget is a slice of the table on which the model cannot reproduce its own answers, identified by a futility stop at the cost of one look. A flip rate that has risen on the same rows against the same cache 55 days later is a model that has moved under it. The certifier reports the movement in the currency the user pays, oracle calls per certified budget. Certified maintenance is therefore useful even where it certifies nothing: the refusal identifies, cheaply, reuse error that would otherwise go unmeasured.

AVAILABILITY AND REPRODUCIBILITY

a: What is released.

The system, the certifier, every experiment script, and every result artifact are public in a single repository, <https://github.com/arjunlohan/project-lore>, under a source-available license that permits research and noncommercial

TABLE 12. Certified savings of the main seed (oracle calls in parentheses) for every configuration and bound; “refused” gives the calls spent before the stop.

Edit	α	exact	Clopper–Pearson	betting	Maurer–Pontil	Bardenet–Maillard
SO formatting-only	0.02	refused (225)	refused (225)	refused (225)	refused (225)	refused (225)
SO formatting-only	0.05	18.7% (1,485)	18.7% (1,485)	18.7% (1,485)	refused (1,485)	refused (1,485)
SO formatting-only	0.10	88.4% (90)	86.2% (135)	86.2% (135)	54.7% (765)	54.7% (765)
SO formatting-only	0.20	88.4% (90)	88.4% (90)	88.4% (90)	86.2% (135)	72.7% (405)
SO synonym rewording	0.02	refused (90)	refused (90)	refused (90)	refused (90)	refused (90)
SO synonym rewording	0.05	refused (90)	refused (90)	refused (90)	refused (90)	refused (90)
SO synonym rewording	0.10	refused (90)	refused (90)	refused (90)	refused (90)	refused (90)
SO synonym rewording	0.20	82.3% (225)	82.3% (225)	82.3% (225)	82.3% (225)	73.3% (405)
SO scope widening	0.02	refused (90)	refused (90)	refused (90)	refused (90)	refused (90)
SO scope widening	0.05	refused (765)	refused (765)	refused (765)	refused (765)	refused (765)
SO scope widening	0.10	72.0% (405)	72.0% (405)	72.0% (405)	18.1% (1,485)	18.1% (1,485)
SO scope widening	0.20	87.8% (90)	87.8% (90)	87.8% (90)	81.0% (225)	72.0% (405)
Djinni formatting-only	0.02	refused (180)	refused (180)	refused (180)	refused (180)	refused (180)
Djinni formatting-only	0.05	refused (315)	refused (315)	refused (315)	refused (315)	refused (315)
Djinni formatting-only	0.10	19.1% (495)	refused (878)	refused (878)	refused (878)	refused (878)
Djinni formatting-only	0.20	52.3% (953)	52.3% (953)	52.3% (953)	19.1% (1,617)	19.1% (1,617)
Djinni criteria change	0.02	refused (180)	refused (180)	refused (180)	refused (180)	refused (180)
Djinni criteria change	0.05	refused (315)	refused (315)	refused (315)	refused (315)	refused (315)
Djinni criteria change	0.10	18.9% (540)	18.9% (540)	18.9% (540)	refused (918)	refused (918)
Djinni criteria change	0.20	62.7% (745)	54.4% (913)	76.3% (475)	27.9% (1,442)	18.9% (1,622)

use. The release contains ingestion scripts for the two corpora, the five versioned edit pairs, the per-cell ground-truth flip labels, the certifier, the adapter conformance suite, and every script and result artifact that produces a number in this article. The corpora are the 2023 Stack Overflow Developer Survey [50] and the English candidate profiles of the Djinni Recruitment Dataset [51]. Both are public and redistributable (under ODbL 1.0/DbCL 1.0 and MIT), and both are used as distributed by their maintainers under those licenses. The Djinni corpus is released anonymized at source, and this work collects no new human-subject data, attempts no re-identification, and releases only per-cell labels and aggregate statistics. Nothing in the evaluation path is withheld. Anyone rerunning the evaluation needs a model endpoint of their own; the labels are released so the statistical results can be reproduced without one.

b: Generated numbers.

Every quantitative claim in this article’s prose, tables, and figures is emitted by a single generator (`scripts/experiments/gen-paper-assets.ts`) from the result artifacts and from the system’s cost ledger, and cited through macros, never typed inline. Re-running an experiment therefore regenerates the sentences that cite it. Several numbers (the per-stratum floors, the ledger economics, the materialization wall clock, the certificate’s stamp count) are computed from the running system’s database, not from a result file. The generator dumps every one of those database reads to an artifact alongside the others, so the release is self-contained, with no dependence on our instance. A checker (`scripts/check-paper-numbers.ts`) enforces the macro discipline mechanically and fails on ten classes of defect, listed in Appendix A; it reports zero of all ten for this manuscript.

c: Hazards and the preprint.

Three reproducibility hazards we hit are documented in the harness, and Section V describes the first two. First, seeded ORDER BY RAND in MySQL depends on the projection list, so evaluation samples are pinned by exact query string. Second, the certifier’s sample path depended on the storage engine’s row order until it was pinned to a canonical order. Third, naive linear-congruential pseudorandom number generators written with JavaScript number multiplication silently lose low bits, so all sampling uses exact 32-bit arithmetic. An earlier version of this article is public as a preprint (Zenodo, doi:10.5281/zenodo.21833641); it is “the preprint” wherever this article says so. Relative to it, the certifier’s default bound changed from Maurer–Pontil to the exact finite-population bound and every number was regenerated. The preprint’s certification results are reproducible from the same release with the Maurer–Pontil arm selected.

APPENDIX A THE MECHANICAL GUARDS

The checker (`scripts/check-paper-numbers.ts`) fails on any of ten conditions:

- 1) a hand-typed quantity in the prose outside an enumerated allow-list (user parameters such as α and δ , protocol constants such as the schedule looks, and generic thresholds quoted as such);
- 2) a quantity hardcoded inside the generator instead of derived from an artifact;
- 3) a macro defined but never cited, since an uncited macro can drift undetected;
- 4) a macro whose trailing space is swallowed into the next word, which is invisible in the source;
- 5) a sentence that asserts two quantities differ while citing two macros whose values are equal, which is how an earlier draft printed “different looks” beside a number repeated twice;
- 6) a spelled-out multiplier (“twice”, “an order of mag-

nitude”) that the cited macros’ ratio does not support, which is how a rate 1.26 times a floor was printed as double it;

- 7) a containment claim (“inside”, “at most”) that the cited range and scalar macros do not support;
- 8) an absolute claim (“never”, “every”, “anywhere”) appearing in the abstract or the contributions list without an entry in an enumerated allow-list, the one class of defect the numeric guards are blind to by construction;
- 9) a scope claim (“on the same column”) whose cited macros come from different columns, checked against a provenance file the generator emits from the same pair definitions the experiments run on; and
- 10) a prose sentence longer than 60 words (tables, captions, numbered lists, displayed mathematics, and the theorem environments excluded), a readability bound.

The generator adds guards of its own. It asserts the relations that Section V and Section VIII state between figures: in the paragraphs on the strict mode, the stratifier ablation, the matched competitor, the family study, the independence check, and the pilot. It also asserts that the separate audit verifier reproduces each audit the article prints. It stops if a regenerated artifact breaks one. Each guard exists because the corresponding defect reached a compiled PDF at least once, several of them in a draft whose numbers were already machine-generated. Generating the quantities is necessary but not sufficient, because the sentences that *relate* quantities go stale independently of the quantities themselves.

APPENDIX B THE BOUND ABLATION, CONFIGURATION BY CONFIGURATION

Table 12 gives, for every configuration of Table 2 and each of the five bounds, the certified savings of the main seed and the oracle calls spent. Relative to the exact bound, the Clopper–Pearson bound certifies all but one of its configurations (lost 1, gained 0), reaching 7 of them at the same look, 0 earlier, and 2 later, since it ignores the finite-population correction. Maurer–Pontil reaches 1 at the same look, 0 earlier, and 6 later. Bardenet–Maillard loses 3 certificates, gains 0, and reaches 0 at the same look, 0 earlier, and 7 later. The betting sequence reaches 7 at the same look, 1 earlier, and 1 later.

ACKNOWLEDGMENT

The author used Claude (Anthropic) as an editing assistant for the manuscript text and as a coding assistant for the system and experiment harness. All technical content, experimental designs, measurements, and conclusions were specified, reviewed, and verified by the author.

REFERENCES

- [1] Google LLC, “Use the AI function in Google Sheets,” Google Docs Editors Help, <https://support.google.com/docs/answer/15877199>, 2026, accessed 29 September 2026.
- [2] Airtable, Inc., “Using Airtable AI in fields,” Airtable Support, <https://support.airtable.com/articles/8052242094-using-airtable-ai-in-fields>, 2026, accessed 29 September 2026.
- [3] Notion Labs, Inc., “Notion AI autofill for databases,” Notion Help Center, <https://www.notion.com/help/autofill>, 2026, accessed 29 September 2026.
- [4] Snowflake Inc., “Snowflake Cortex AI functions: AI_COMPLETE and AI_CLASSIFY,” Snowflake Documentation, https://docs.snowflake.com/en/sql-reference/functions/ai_complete, 2026, accessed 29 September 2026.
- [5] Microsoft Corporation, “COPILOT function,” Microsoft Support, <https://support.microsoft.com/en-us/excel/functions/copilot-function>, 2026, preview feature, withdrawn 14 September 2026; accessed 29 September 2026.
- [6] S. Ouyang, J. M. Zhang, M. Harman, and M. Wang, “An empirical study of the non-determinism of ChatGPT in code generation,” *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 42:1–42:28, 2025, arXiv:2308.02828, doi:10.1145/3697010.
- [7] B. Atil, S. Aykent, A. Chittams, L. Fu, R. J. Passonneau, E. Radcliffe, G. R. Rajagopal, A. Sloan, T. Tudrej, F. Ture, Z. Wu, L. Xu, and B. Baldwin, “Non-determinism of ‘deterministic’ LLM system settings in hosted environments,” in *Proc. 5th Workshop on Evaluation and Comparison of NLP Systems*, 2025, pp. 135–148, doi:10.18653/v1/2025.eval4nlp-1.12.
- [8] J. Yuan, H. Li, X. Ding, W. Xie, Y.-J. Li, W. Zhao, K. Wan, J. Shi, X. Hu, and Z. Liu, “Understanding and mitigating numerical sources of nondeterminism in LLM inference,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2506.09501.
- [9] H. He and Thinking Machines Lab, “Defeating nondeterminism in LLM inference,” Thinking Machines Lab: Connectionism, technical report, 2025, accessed 15 August 2026, doi:10.64434/tml.20250910.
- [10] D. Kang, E. Gan, P. Bailis, T. Hashimoto, and M. Zaharia, “Approximate selection with guarantees using proxies,” *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 1990–2003, 2020, doi:10.14778/3407790.3407804.
- [11] C. J. Clopper and E. S. Pearson, “The use of confidence or fiducial limits illustrated in the case of the binomial,” *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934, doi:10.1093/biomet/26.4.404.
- [12] A. Maurer and M. Pontil, “Empirical Bernstein bounds and sample variance penalization,” in *Proc. Conf. on Learning Theory (COLT)*, 2009, online proceedings, paper 012; also arXiv:0907.3740.
- [13] R. Bardenet and O.-A. Maillard, “Concentration inequalities for sampling without replacement,” *Bernoulli*, vol. 21, no. 3, pp. 1361–1385, 2015, doi:10.3150/14-BEJ605.
- [14] I. Waudby-Smith and A. Ramdas, “Estimating means of bounded random variables by betting,” *J. R. Stat. Soc. Series B*, vol. 86, no. 1, pp. 1–27, 2024, doi:10.1093/jrsssb/bkqad009.
- [15] S. R. Howard, A. Ramdas, J. McAuliffe, and J. Sekhon, “Time-uniform, nonparametric, nonasymptotic confidence sequences,” *Ann. Statist.*, vol. 49, no. 2, pp. 1055–1080, 2021, doi:10.1214/20-AOS1991.
- [16] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, and L. Lei, “Learn then test: Calibrating predictive algorithms to achieve risk control,” *Ann. Appl. Stat.*, vol. 19, no. 2, pp. 1641–1662, 2025, arXiv:2110.01052, doi:10.1214/24-AOAS1998.
- [17] F. Gilardi, M. Alizadeh, and M. Kubli, “ChatGPT outperforms crowd workers for text-annotation tasks,” *Proc. Natl. Acad. Sci. USA*, vol. 120, no. 30, p. e2305016120, 2023, doi:10.1073/pnas.2305016120.
- [18] C. Ziems, W. Held, O. Shaikh, J. Chen, Z. Zhang, and D. Yang, “Can large language models transform computational social science?” *Comput. Linguist.*, vol. 50, no. 1, pp. 237–291, 2024, doi:10.1162/coli_a_00502.
- [19] N. Pangakis, S. Wolken, and N. Fasching, “Automated annotation with generative AI requires validation,” arXiv preprint arXiv:2306.00176, 2023.
- [20] P. Törnberg, “ChatGPT-4 outperforms experts and crowd workers in annotating political Twitter messages with zero-shot learning,” arXiv preprint arXiv:2304.06588, 2023.
- [21] F. Bang, “GPTCache: An open-source semantic cache for LLM applications enabling faster answers and cost savings,” in *Proc. 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS)*, 2023, pp. 212–218, doi:10.18653/v1/2023.nlpss-1.24.
- [22] R. J. Serfling, “Probability inequalities for the sum in sampling without replacement,” *Ann. Statist.*, vol. 2, no. 1, pp. 39–48, 1974, doi:10.1214/aos/1176342611.
- [23] J. Lao, A. Zimmerer, O. Ovcharenko, T. Cong, M. Russo, G. Vitagliano, M. Cochez, F. Özcan, G. Gupta, T. Hottelier, H. V. Jagadish, K. Kissel, S. Schelter, A. Kipf, and I. Trummer, “SemBench: A benchmark for semantic query processing engines,” *Proc. VLDB Endow.*, vol. 19, no. 8, pp. 1754–1767, 2026, doi:10.14778/3811243.3811249.
- [24] D. Kang, J. Guibas, P. Bailis, T. Hashimoto, Y. Sun, and M. Zaharia, “Accelerating approximate aggregation queries with expensive predicates,” *Proc. VLDB Endow.*, vol. 14, no. 11, pp. 2341–2354, 2021, doi:10.14778/3476249.3476285.

- [25] S. Zeighami, S. Shankar, and A. Parameswaran, "Cut costs, not accuracy: LLM-powered data processing with guarantees," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–26, 2025, SIGMOD 2026, doi:10.1145/3769776.
- [26] L. Patel, S. Jha, M. Pan, H. Gupta, P. Asawa, C. Guestrin, and M. Zaharia, "Semantic operators and their optimization: Enabling LLM-based data processing with accuracy guarantees in LOTUS," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4171–4184, 2025, doi:10.14778/3749646.3749685.
- [27] S. Jo and I. Trummer, "ThalamusDB: Approximate query processing on multi-modal data," *Proc. ACM Manag. Data*, vol. 2, no. 3, pp. 1–26, 2024, doi:10.1145/3654989.
- [28] S. Shankar, S. Zeighami, and A. Parameswaran, "Task cascades for efficient unstructured data processing," *Proc. ACM Manag. Data*, vol. 4, no. 1, pp. 1–26, 2026, SIGMOD 2026, doi:10.1145/3786702.
- [29] L. G. Schroeder, A. Desai, A. Cuadron, K. Chu, S. Liu, M. Zhao, S. Krusche, A. Kemper, M. Zaharia, and J. E. Gonzalez, "vCache: Verified semantic prompt caching," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2026, arXiv:2502.03771.
- [30] M. Mansoor, T. Ahmad, and Y.-C. Yoon, "Risk-constrained freshness-aware semantic caching for open-web retrieval-augmented LLMs," arXiv preprint arXiv:2607.04281, 2026.
- [31] A. W. Lee, B. Han, S. Sen, S. Yeom, U. Çetintemel, and A. Datta, "Evergreen: Efficient claim verification for semantic aggregates," arXiv preprint arXiv:2604.26180, 2026.
- [32] S. Krishnan, J. Wang, M. J. Franklin, K. Goldberg, and T. Kraska, "Stale view cleaning: Getting fresh answers from stale materialized views," *Proc. VLDB Endow.*, vol. 8, no. 12, pp. 1370–1381, 2015, doi:10.14778/2824032.2824037.
- [33] J. M. Hellerstein, P. J. Haas, and H. J. Wang, "Online aggregation," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 1997, pp. 171–182, doi:10.1145/253260.253291.
- [34] M. Budiu, T. Chajed, F. McSherry, L. Ryzhyk, and V. Tannen, "DBSP: Automatic incremental view maintenance for rich query languages," *Proc. VLDB Endow.*, vol. 16, no. 7, pp. 1601–1614, 2023, doi:10.14778/3587136.3587137.
- [35] A. Gupta, I. S. Mumick, and K. A. Ross, "Adapting materialized views after redefinitions," in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, 1995, pp. 211–222, doi:10.1145/223784.223817.
- [36] A. Nouri, "StepCache: Step-level reuse with lightweight verification and selective patching for LLM serving," arXiv preprint arXiv:2603.28795, 2026.
- [37] S. Shankar, H. Li, P. Asawa, M. Hulsebos, Y. Lin, J. D. Zamfirescu-Pereira, H. Chase, W. Fu-Hinthorn, A. G. Parameswaran, and E. Wu, "SPADE: Synthesizing data quality assertions for large language model pipelines," *Proc. VLDB Endow.*, vol. 17, no. 12, pp. 4173–4186, 2024, doi:10.14778/3685800.3685835.
- [38] T. Dixit, D. Lee, S. Fang, S. S. Harsha, A. Sureshan, A. Maharaj, and Y. Li, "RETAIN: Interactive tool for regression testing guided LLM migration," in *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 2024, pp. 301–310, arXiv:2409.03928, doi:10.18653/v1/2024.emnlp-demo.31.
- [39] S. Bates, A. Angelopoulos, L. Lei, J. Malik, and M. I. Jordan, "Distribution-free, risk-controlling prediction sets," *J. ACM*, vol. 68, no. 6, pp. 43:1–43:34, 2021, doi:10.1145/3478535.
- [40] C. Liu, M. Russo, M. Cafarella, L. Cao, P. B. Chen, Z. Chen, M. Franklin, T. Kraska, S. Madden, R. Shahout, and G. Vitagliano, "Palimpsest: Optimizing AI-powered analytics with declarative query processing," in *Proc. Conf. on Innovative Data Systems Research (CIDR)*, 2025, <https://vldb.org/cidrdb/papers/2025/p12-liu.pdf>, accessed 30 September 2026.
- [41] S. Shankar, T. Chambers, T. Shah, A. G. Parameswaran, and E. Wu, "DocETL: Agentic query rewriting and evaluation for complex document processing," *Proc. VLDB Endow.*, vol. 18, no. 9, pp. 3035–3048, 2025, doi:10.14778/3746405.3746426.
- [42] S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, "Language models enable simple systems for generating structured views of heterogeneous data lakes," *Proc. VLDB Endow.*, vol. 17, no. 2, pp. 92–105, 2023, doi:10.14778/3626292.3626294.
- [43] H. Dai, B. Y. Wang, X. Wan, B. Dai, S. Yang, A. Nova, P. Yin, P. M. Phothilimthana, C. Sutton, and D. Schuurmans, "UQE: A query engine for unstructured databases," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2407.09522.
- [44] S. Chen, J. Liu, D. Raghavan, and U. Çetintemel, "VectraFlow: Long-horizon semantic processing over data and event streams with LLMs," arXiv preprint arXiv:2604.03855, 2026.
- [45] S. Chen, D. Raghavan, and U. Çetintemel, "Continuous prompts: LLM-augmented pipeline processing over unstructured streams," arXiv preprint arXiv:2512.03389, 2025.
- [46] P. Liskowski and K. Schmaus, "Streaming model cascades for semantic SQL," arXiv preprint arXiv:2604.00660, 2026.
- [47] D. M. L. Martins and G. Vossen, "Prompt as a data type: In-database LLM prompt management and rewriting," arXiv preprint arXiv:2607.21756, 2026.
- [48] U. Çetintemel, S. Chen, A. W. Lee, D. Raghavan, D. Lu, and A. Crotty, "Making prompts first-class citizens for adaptive LLM pipelines," in *Proc. Conf. on Innovative Data Systems Research (CIDR)*, 2026, arXiv:2508.05012.
- [49] R. Wang and A. Ramdas, "False discovery rate control with e-values," *J. R. Stat. Soc. Series B*, vol. 84, no. 3, pp. 822–852, 2022, doi:10.1111/rssb.12489.
- [50] Stack Overflow, "Stack Overflow annual developer survey 2023," <https://survey.stackoverflow.co/2023/>, 2023, accessed 1 October 2026.
- [51] N. Drushchak and M. Romanishyn, "Introducing the Djinni recruitment dataset: A corpus of anonymized CVs and job postings," in *Proc. 3rd Ukrainian Natural Language Processing Workshop (UNLP) at LREC-COLING 2024*, 2024, pp. 8–13, doi:10.63317/2dcvy45ws6yb.

ARJUN LOHAN received the B.S. degree in chemical engineering from the University of California San Diego, La Jolla, CA, USA, in 2019, and the M.S. degree in financial engineering from the University of Southern California, Los Angeles, CA, USA, in 2024, and is currently affiliated with the University of Southern California, where this work was carried out. Research interests include data systems for language-model pipelines, semantic caching and materialized-view maintenance, and statistical guarantees for machine-computed data.

...